Module 2. Robotic Electronics

Overview

Learning Outcomes:

Upon completion of this module, students will:

- Understand basic electronics within the context of robotics.
- Establish hands-on technical competency to build simple circuits utilized in a robotics context.
- Develop trust in the components comprising a robotics system and their functions within a robot.

What to Expect:

This module covers basic electronics from an introductory perspective. You'll learn the relationship between voltage, current, and resistance, and how to calculate each using Ohm's law. You'll learn to read and interpret basic circuit diagrams and be able to discuss concepts of series and parallel circuits.

We'll discuss hardware design, how to choose various components, and how to safely assemble and wire those components together and with on-board computers. We will continue to build a cycle of trust by developing an understanding of the underlying electronics that comprise robotics systems, their interconnection, and their role in the overall system's function, capabilities, and limitations.

During the lab at the end of the module, you will gain some experience designing your own simple circuits using the online simulator TinkerCAD. You'll then build your circuits and test them with electronic tools such as multimeters and oscilloscopes.

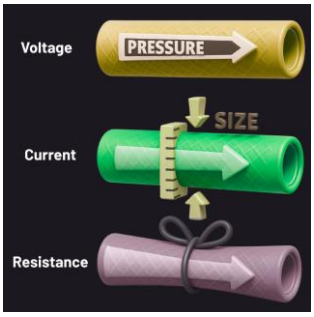
Materials:

Table of Contents:

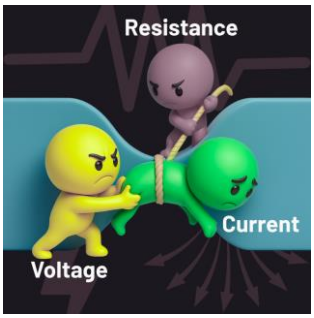
Foundations of Electrical Systems

Lecture 1.1 Voltage, Current, Resistance, and Power

The Role of Electricity in Robotic Systems



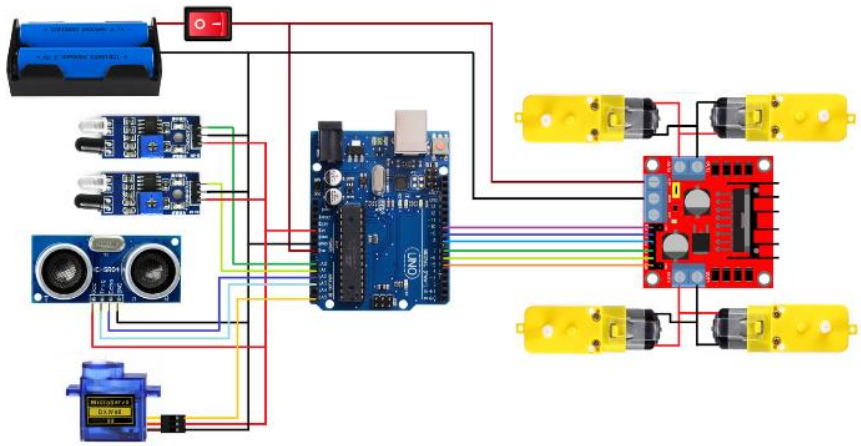
Robots are powered by electricity; not just to move, but to sense, compute, and communicate. Every sensor that perceives, every processor that calculates, and every motor that spins into action relies on electrical energy flowing through a carefully designed system of circuits. In Module 1, we explored how robots operate in a *perceive-process-act* cycle. In this module, we'll uncover the electrical principles that make that cycle possible.



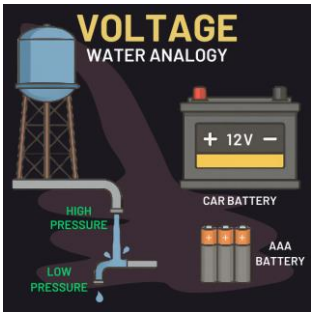
At the core of these electrical systems are three basic principles: *voltage*, *current*, and *resistance*. Together, they determine how electrical energy moves, how much is delivered, and what limits its flow. These concepts also form the basis for calculating power (i.e., the rate at which energy is consumed in a circuit). Every robotic action, from reading a sensor to powering a wheel, depends on the interaction of these forces.

Understanding these fundamentals allows you to design circuits that deliver the right amount of power to the right components at the right time. Whether you're choosing a resistor for an LED, setting power levels for a motor driver, or debugging why a sensor is unresponsive, it all comes back to these same principles.

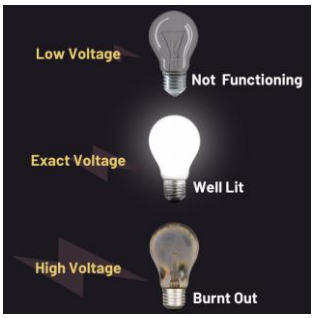
This lecture provides a working understanding of voltage, current, and resistance and shows how they are applied in the circuits that make robotic systems possible.



Voltage: The Driving Force



Voltage is the electrical pressure that pushes electric charges through a circuit. It represents the potential energy per unit charge available to move electrons from one point to another. You can think of voltage like the pressure in a water pipe: the higher the pressure, the more forcefully the water can flow. In the same way, higher voltage means more energy is available to push electric current through a circuit.



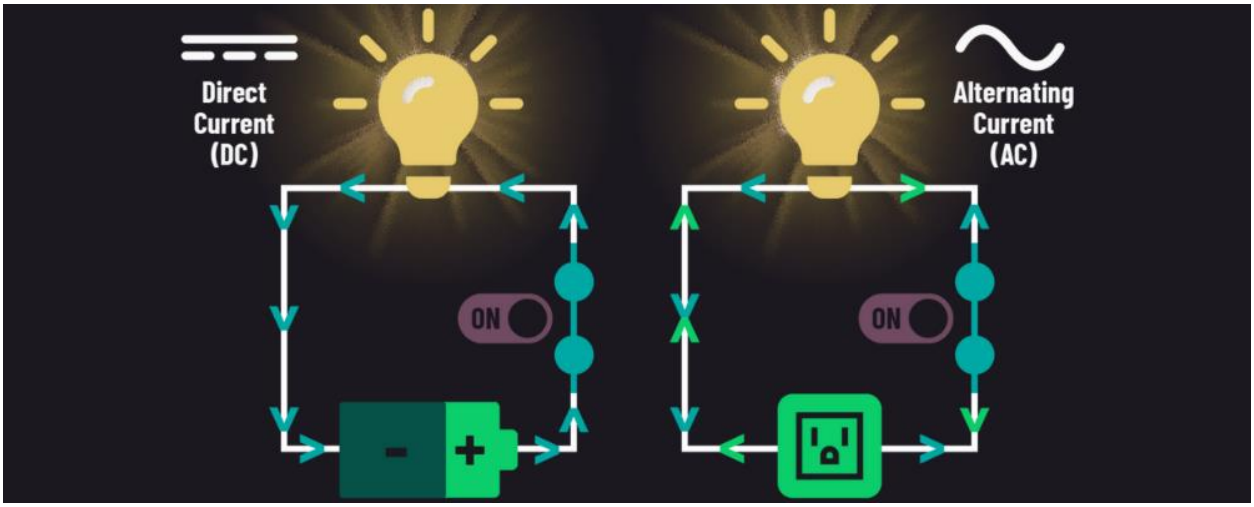
Voltage, measured in volts (V), is often described as electromotive force (EMF), because it drives the movement of electrons. In equations, voltage is commonly represented by the capital letters V or E, especially in formulas like Ohm's Law (which we'll go over later in this lecture). It is a measure of the difference in electric potential between two points in a circuit. If there is no voltage difference, there is no driving force to move a charge, and therefore, no current flows; the circuit does not function.

Current: The Flow of Charge

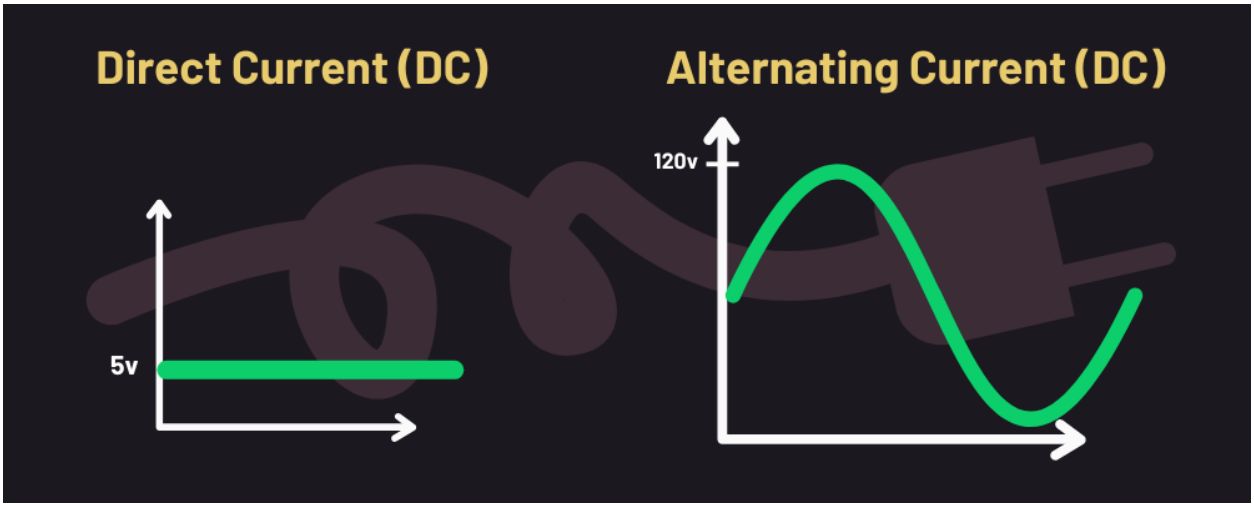
Current is the flow of electric charge through a circuit, driven by voltage, and is measured in amperes (A). It is current that does work, not voltage. While voltage is potential energy, current is kinetic energy. It might seem logical for the symbol that represents electric current to be "C" (for current) or even "A" (for amps), the standard symbol is actually the capital letter I. This comes from the French term "*intensite du courant*," (Intensity of current) convention established by early European physicists.

For current to flow, a complete, closed-loop circuit must be present, consisting of at least a power source with positive and negative terminals. For a circuit to be a closed loop, there must be a path from the positive to the negative potential. There are two primary types of

electrical current:



- Direct current (DC) flows continuously in a single direction. It is typically supplied by batteries or power supplies and flows from the positive terminal to a negative terminal in terms of conventional current. In reality, electrons, which are negatively charged, move from negative to positive, but conventional flow (positive to negative) remains the standard in most texts.
- Alternating Current (AC) periodically reverses its direction. This is the type of current provided by standard household outlets. AC is typically represented by a sine (sinusoidal) wave, which shows the voltage rising from zero to a maximum in one direction (positive), returning to zero, then reaching a maximum in the opposite direction (negative), and returning to zero again. One complete oscillation is called a cycle, and the number of cycles per second is the frequency of the power, measured in hertz (Hz).

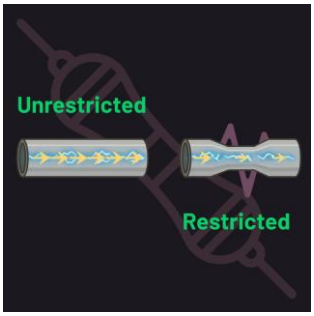


Going back to the water pipe analogy:

The tank provides water pressure, just like a battery provides voltage (both are potential energy). The amount of water flowing through the pipe each second represents electric current. If the faucet is nearly closed, you will see a small stream of water; that would be like low current flow; only a small amount of electrical current is flowing. If the faucet is opened up you will see a gush of water, that would be like high current flow; a large amount of electrical current is moving through the circuit.

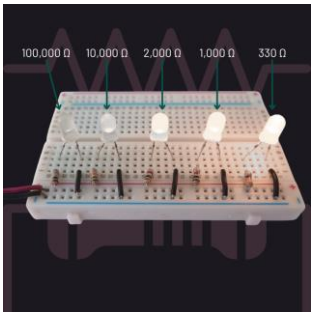
Resistance: Controlling the Flow

Resistance controls the amount of current that flows through a circuit by opposing the flow of electrons, effectively reducing the current flow. It's like electrical friction: the more resistance, the harder it is for the current to move through the circuit. It is represented by the capital letter R and is measured in ohms (Ω).



In every circuit, resistance plays a critical role in protecting sensitive components from damage. You can think of resistance like a narrow section or valve in a water pipe. If there is a section of the pipe that is compressed or partially closed, the water flow is restricted, allowing less water to be pushed through. Similarly, resistance restricts the electrical flow in the circuit.

Every power-consuming component is designed to draw a specific amount of current. If a component receives too much current, it can be damaged. If it receives too little, it may not function correctly or at all. For example, a light-emitting diode (LED) can burn out if it draws too much current but may not glow at all if the current is too low.

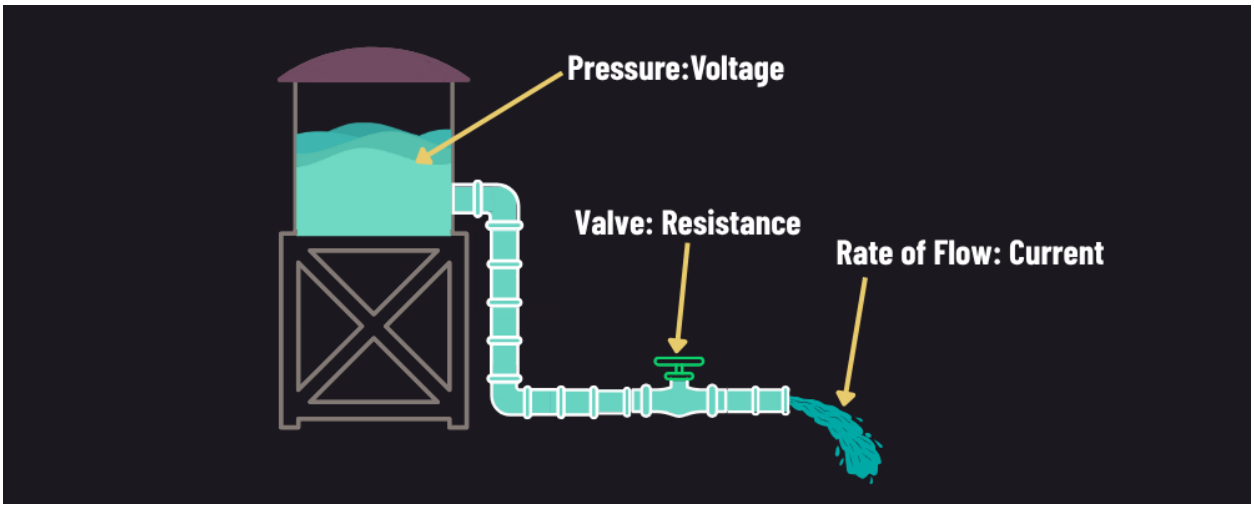


In most practical circuits, we deal with higher available current, so we use resistors with specific values to reduce the current to a safe and functional level. This ensures that each component works as intended, such as an LED shining without burning it out.

The Relationship between Voltage, Current, and Resistance

Let's put these concepts together now using the water analogy from earlier. Understanding this relationship helps explain how circuits respond to changes in voltage, resistance, or current:

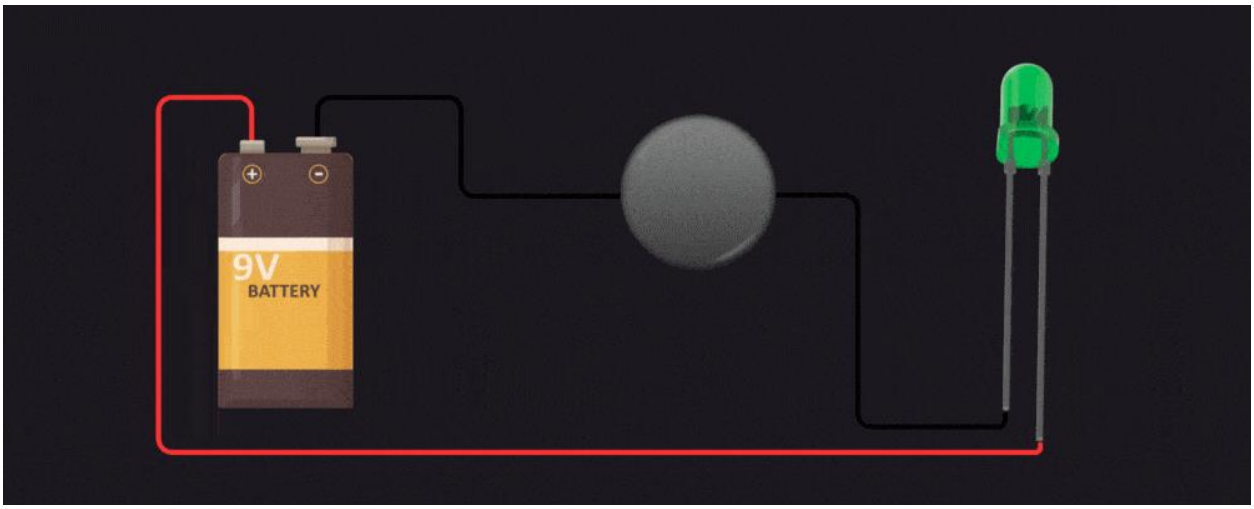
- The height of the water represents voltage: more height means more water pressure caused by gravity. The more water, the more potential energy it has.
- The width of the pipe represents resistance. A narrow pipe restricts flow (high resistance), while a wider pipe allows more water to pass (low resistance).
- The flow of water through the pipe is like electric current: how much water moves through per second.



Lecture 1.2 Circuit Design and Behavior

What is a Circuit?

Before we can design and build electronic devices, we need to understand what a circuit is. A circuit is a complete, closed loop that allows electrical current to flow. If the loop is broken, current cannot flow, and the circuit won't function. If the loop is complete, the electric current flows, powering the components within it.



To create a circuit, you need a few basic parts:

- A power source (e.g., a battery or power supply)
- A load that uses electricity (like a motor, sensor, or LED)
- Conductive pathways —typically wires or traces— that carry current between the components.
- Often a controller, such as a switch to regulate when the current flows.

The purpose of a circuit is to guide electricity so it can do something useful. This might be turning on a light, running a fan, or powering a computer. Circuits are the foundation of all electronic and robotic systems. Understanding how they work is the first step toward building more complex machines.

Electrical Components and Circuit Behavior

For a circuit to do work, it needs something to power. This is called a "load", which is a component that consumes electrical energy and converts it into useful work. This could be a motor that spins, a light bulb that glows, a sensor that detects motion, or a heater that warms a room. Loads come in different types, and each one interacts with electrical current differently. The three main categories of interaction are resistive, inductive, and capacitive loads.

RESISTIVE LOADS generate heat and/or light when current flows through them. Heat is generated when the resistive load opposes current flow. For example, resistors oppose current flow, and when there is enough current, they can get hot. An incandescent light bulb is another common example, where resistance in the filament causes it to glow and produces heat.

INDUCTIVE LOADS are devices that use current to generate a magnetic field, such as motors, transformers, solenoids, and speakers. A defining property of inductance is that it resists sudden changes in current — when current flowing through an inductive load changes, the inductor produces a voltage that opposes the change. This makes inductive loads useful in systems involving motion or sound. However, when the current is interrupted (such as when the device is switched off), the collapsing magnetic field can generate high-voltage spikes that may damage components unless properly managed. This property is useful for systems involving movement or sound, such as speakers and fans. However, they can also introduce voltage spikes when turned off.

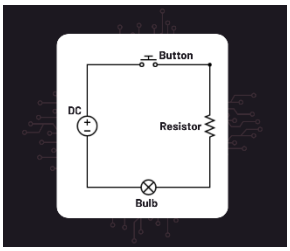
CAPACITIVE LOADS store energy in an electric field similar to a battery using components called capacitors. In DC circuits, a capacitor initially draws current while charging, but once fully charged, it blocks further current flow. In AC circuits, capacitive loads can pass alternating signals and affect how current and voltage behave over time, often causing the current to lead the voltage. Capacitive loads are common in applications like sensors, power converters, and HVAC systems, where energy storage, filtering, and timing are important design factors.

Each load type affects the circuit differently. Capacitive loads initially look like a short and draw a lot of current at startup, while inductive loads initially look like an open until the magnetic field is created. Capacitive loads resist changes in voltage, while inductive loads resist changes in current. Resistive loads release energy as heat and/or light, which may require thermal management. Inductive loads can create electrical noise or voltage spikes when magnetic fields are created and collapsed that may affect other components in a circuit, requiring shielding.

When designing robotic systems, it's important to select components appropriate to the circuit's function and ensure that they are compatible with your power source and control systems.

Circuit Symbols

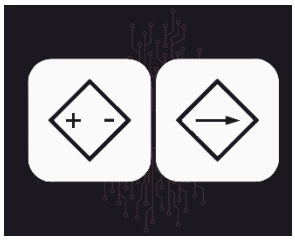
Standardized Symbols: When engineers and technicians design or troubleshoot electronics, they use schematic diagrams to communicate how a circuit is constructed. These diagrams rely on standardized symbols to represent different electrical components – especially in robotics, where complex circuits are common.



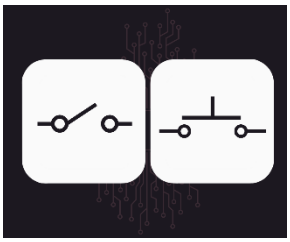
Wires: The simplest of symbols, the wires, cables, and conductive paths in a circuit are represented by a straight solid line. It connects components and allows current to flow between components.

Independent Power Sources: Independent power sources provide a constant voltage and current. These power sources are represented by circles that contain a symbol indicating the type of output. An AC voltage source has a curved line (i.e., sine wave), DC voltage source shows plus and minus symbols (i.e., positive and negative terminals). You also may see an arrow in a circle representing the DC current flow. Batteries are represented by stacked long and shorter lines. The long line on one side is positive, the short line on the opposite side is negative.

Dependent Power Sources: These schematic symbols represent components used in electrical engineering known as controlled sources. The symbol on the left, which features plus and minus signs inside a diamond, indicates a voltage output that is dependent upon the input control voltage or current. The symbol on the right, which contains an arrow inside a diamond, signifies a current output and whose output depends upon an input controlling voltage or current. Determining whether each source is controlled by a voltage or a current signal requires additional context from the schematic, such as input connections or labeling.

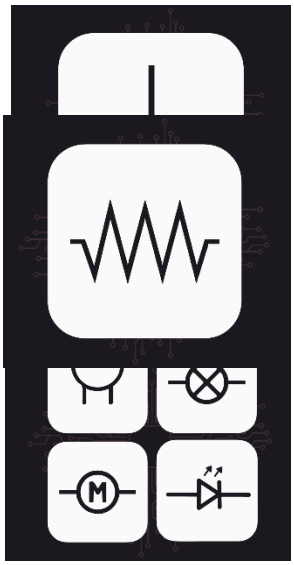


Switches: Switches control the flow of current by opening or closing the circuit path. The most basic symbol looks like a break in the wire with a lever angled toward it. Similar to an open gate on a fence. These are examples of very simple switches. In practice, there are a lot of different types of switches, many of which are complex in their operation.



Ground: Ground provides a common return path for current and serves as a voltage reference point. The ground symbol usually consists of three parallel, horizontally stacked

lines. They are each a different width with the longest one being closest to the circuit farthest being the shortest.



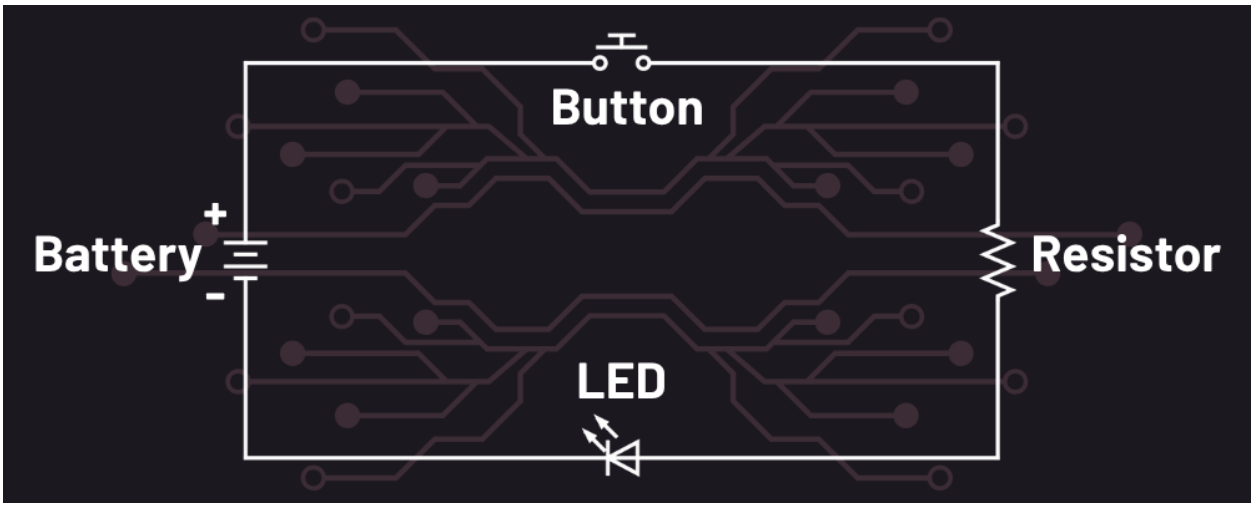
Resistors: The resistor symbol is a zigzag line. Each end of the zigzag is attached to the circuit. Recall that resistor limit is the amount of current flow in a circuit and is critical for protecting sensitive components.

Loads: Here are some schematic symbols for common load devices. A light bulb is represented as a circle with an X or loop inside it. A buzzer is an audio device that is represented as a half-circle with two connectors attaching it to the circuit from its curved side. A motor is depicted as a circle with the capital letter M inside it. A light emitting diode (LED) is depicted as a triangle, followed by an attached vertical straight line. Two small arrows indicate that it is a light emitting diode as opposed to a standard diode.

Reading and Interpreting Schematic Diagrams

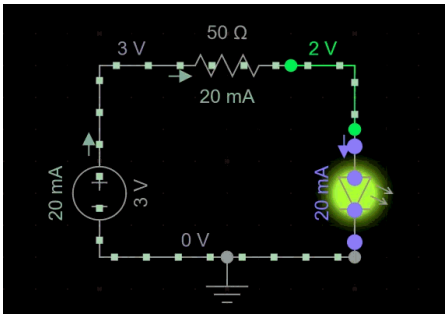
When designing or troubleshooting circuits, it's important to look beyond the physical layout of wires and components. Engineers use schematic diagrams (also called circuit diagrams) which are abstract representations of electronic circuits to clearly and concisely communicate how a system is wired, the components, and how it operates. Schematics are not drawn to physical scale or layout; instead, they show logical connections between components using standard circuit symbols.

Schematic diagrams allow engineers and technicians to share and interpret designs, regardless of manufacturer or application. Schematics are used by engineers and technicians in circuit manufacturing, debugging, and routine maintenance.



For example, a simple circuit might include a power source connected to a switch, which then connects to a resistor, followed by an LED, and finally to ground. This layout shows the conventional flow of current from the positive terminal of the power source, through each component, and back to the negative terminal (often labeled as ground). Each component in this circuit plays a specific role. The switch controls the flow of current. The resistor limits the amount of current in the circuit protecting the LED. The LED is the load, converting electrical energy into light. Ground is the return path for the current, completing the circuit.

Understanding how to read and interpret schematics is a foundational skill in electronics. As you work through labs and projects, take the time to sketch out or read circuit diagrams before you start connecting parts. It will save you time and help prevent mistakes.

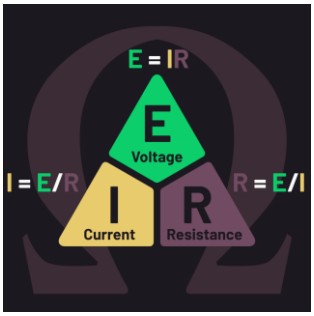


Here are some basic steps to follow when reading a schematic:

1. Follow the flow from power to ground.
2. Identify symbols using the reference guide (from the previous section).
3. Label currents and voltages to predict how energy will flow and what values to expect.
4. Compare to a circuit simulator or real-world behavior when building or troubleshooting.

Lecture 1.3 Analyzing Circuits

Ohm's Law – The Relationship



Ohm's Law describes the relationship between voltage (E), current (I), and resistance (R), and is one of the most fundamental principles in electronics. The law is expressed as:

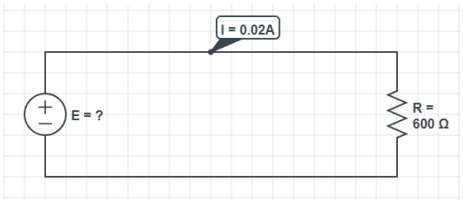
$$E = I \times R$$

(Voltage = Current x Resistance)

*(Volts = amps * ohms)*

This equation shows how these three quantities interact in any electrical circuit. If you know any two of the values, you can calculate the third.

For example, suppose we want to calculate the voltage needed to supply 20 mA (0.02A) of current across a 600-ohm (600Ω) resistor. Using the equation above:

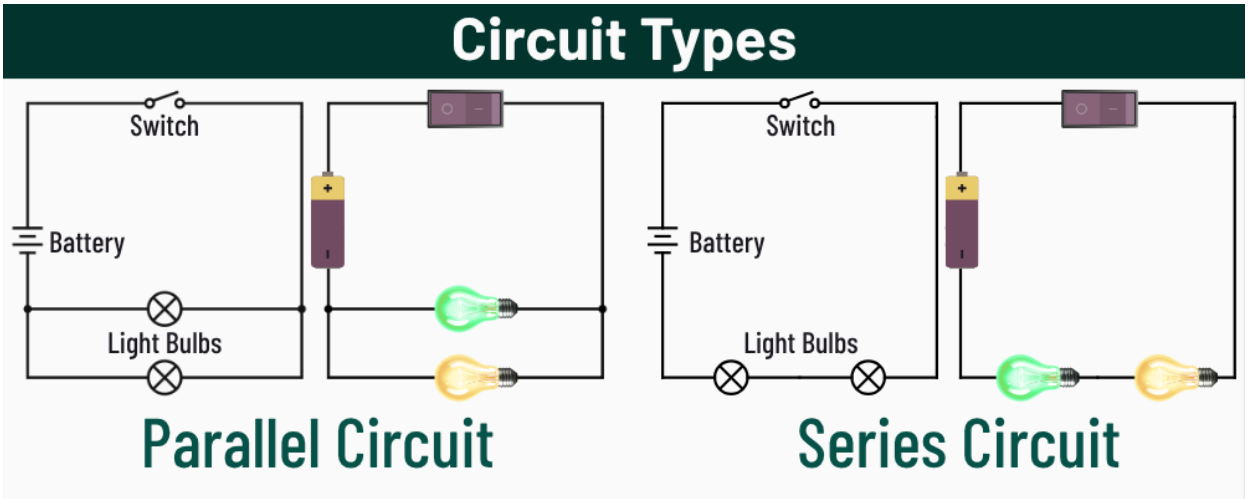


$$E = I \times R$$

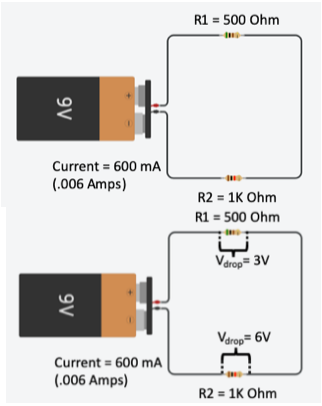
$$E = 0.02 \times 600 = 12 \text{ volts}$$

Series vs. Parallel

Now that we've covered what a circuit is, let's look at two common ways of organizing components in a circuit: series circuits and parallel circuits. These layouts determine how electricity flows through a system, and understanding the difference is essential to building functional, reliable robotics systems.



In a series circuit, all the components are connected one after another in a single path. This means the same current flows through every part of the circuit. If there is a break anywhere in the circuit, the current stops flowing in the entire circuit. Imagine a string of lights where one broken bulb causes the entire strand to go dark. The total voltage across the circuit is divided among the components based on each component's resistance and the current flowing through the circuit. Ultimately, the sum of the voltages of each individual component should add up to the total voltage supplied by the power source.

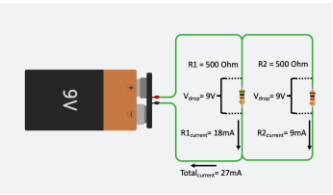


Let's imagine a series circuit with a 9V power supply and two resistors. One resistor is 500 ohms, another resistor is 1K ohms, and there are 6 milliamps of current flowing through the circuit as shown below:

Voltage drops across components are computed by multiplying current by the component resistance. So, the first resistor (R1) will drop 500 ohm × 6 milliamps, or 3V. The second resistor (R2) will drop 1K ohm × 6 milliamps, or 6V. Again, the sum of the voltage drops should equal the source voltage. In our example, 3V drops across R1, plus a 6V drops across R2, for a total of 9V, which is the supply voltage. This is illustrated in the image below:

In a parallel circuit, the components are connected across multiple paths with respect to the power supply, each forming its own branch. In this layout, each branch receives the same voltage from the power source, and the current is divided among the branches (essentially the opposite of a series circuit). This means that if one branch of a parallel circuit stops working, the current continues to flow to the other branches. The current from the inoperative branch is now distributed among the other branches, which can exceed the tolerances of the components in those other branches causing damage to the system.

Let's apply a parallel circuit to our example. Suppose we have a 9V power supply, with the same two resistors from above, but in parallel.



Because this is a parallel circuit, the voltage drop across both resistors is the same (9v). However, the current in each branch will be different. Current across each resistor is computed by dividing the voltage (9v) by the resistance. So, the current through the first resistor (R1) is 9 volts/500 ohms, or 18 milliamps. The current through the second resistor (R2) is 9 volts/1K ohms, or 9 milliamps. The total current flowing through the system is the sum of the current in the two branches, I1 + I2 = 18 + 9 = 27 milliamps.

Calculating Resistance and Current

Hopefully by now you have a foundational understanding of the structure and behavior of circuits. Let's apply Ohm's Law to calculate how voltage, current, and resistance behave in real-world circuits.

Recall Ohms Law from the previous lecture: E = I x R (Voltage = Current x Resistance). Using this formula, you can solve for any one of the three values, as long as you know the other two.

Example 1: Calculate Needed Resistance

Let's say you're using a 5V power source to light two indicator lights connected one after another in a series circuit. You know from a previous section that in a series circuit, voltage is divided across components. When current flows through a component, it resists current flow while it does work (light, motor, resistor) and creates a difference in potential across the component called a voltage drop. So, first consider how much voltage is being dropped by all the components and compare that to how much voltage is available from the power supply.

1. Sum up all the voltage expected to be required. In this case, each light needs 2V.

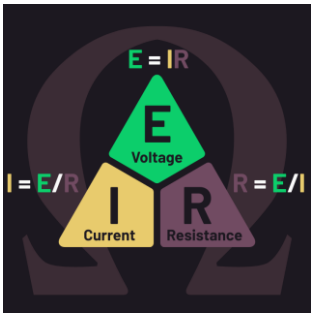
   2V + 2V = 4V required

2. Calculate the remaining voltage from the power supply. In this case, only 4V will be dropped out of the 5V from the power supply. The remaining 1 V must be dropped somewhere on the circuit to avoid damaging circuit components.

   remaining voltage: 5V – 4V = 1V

3. Consider the current flow in the circuit. Recall that each component requires a specific amount of current to operate safely. In this case, the recommended current for each light is 20mA (0.02 A).

4. Use Ohm's Law to find the resistor value. You know from Ohm's Law that energy (voltage) = current (amperes) * Resistance (ohms) you can solve for resistance by dividing the current on both sides of the equation.



$$E = I * R$$

$$R = E / I$$

needed resistance = unused voltage / amperage for the component

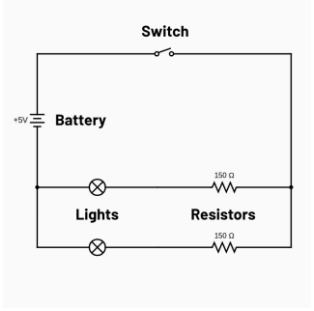
needed resistance = 1V / 0.02A

needed resistance = 50 ohms

From this calculation, you have determined that you need a resistor of at least 50Ω. However, if there is not a resistor of that value, you would find a resistor of the nearest value, without going under 50Ω (typically 100Ω). Going under could allow too much current and damage the lights.

Example 2: Calculate Total Current Draw

Now imagine a parallel circuit with two lights, each using about 2V, each connected through its own 150Ω resistor, both sharing the same 5V power source.



1. Consider the voltage drop. In a parallel circuit, the same voltage is applied to each branch. If each light uses 2V, the remaining voltage across the resistor is 3V.
2. Consider the amount of resistance present. Within one branch, there is 150Ω resistor in series with the light to absorb the remaining voltage.
3. Find current through one branch. You know that from energy (voltage) = current (amperes) * Resistance (ohms) you can solve for current by dividing the resistance on both sides of the equation.

$$E = I * R$$

$$I = E / R$$

current for one branch = unused voltage / resistance value

current for one branch = 3V / 150Ω

current for one branch = 0.02A (20 mA)

4. Find the total current draw for all (both) branches. To do this, you simply need to add the current for every branch in the circuit.

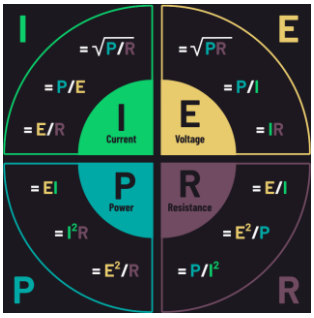
total current draw = branch one current + branch two current

total current draw = 0.02 A + 0.02 A

total current draw = 0.04 A (40 mA)

From this calculation, you have determined that your power supply needs to provide at least 0.04 A (40mA) of current.

Power



Electrical power is the rate at which energy is transferred or converted in a circuit. It tells us how much work electricity can do over time, such as generating heat, light, or motion. Power is measured in watts (W) and depends on both voltage (E) and current (I).

In mechanical systems, work involves moving an object against a resisting force. In electrical systems, work is done when electrons move through resistance, which requires energy. The more energy transferred per second, the greater the power.

The most common equation used to calculate power is:

P = E x I

(Power = Voltage x Current)

(Watts = Volts x Amps)

Depending on the known values in a circuit, these alternative formulas can also be used:

- $P = E^2 / R$ (Power = Voltage squared divided by resistance)
- $P = I^2 \times R$ (Power = Current squared times resistance)

These formulas describe the same principle from different perspectives. Based on the above formulas, one amp of current through one ohm of resistance is one watt of power. Understanding this relationship is essential for designing circuits that are safe and efficient.

In many engineering and electronics applications, especially in embedded systems, designers must consider SWaP: Size, Weight, and Power. Minimizing power usage not only

improves efficiency but also reduces the size and heat output of on-board devices, which can be critical in aerospace, robotics, and wearable applications.

Interpreting Power Sources and Labels

Every power source provides a specific voltage and has a maximum current capacity. For example, on the side of a "AAA" battery, you might see the numbers 1.2V and 600mAh (mAh stands for milli amp hours). This means the battery can supply 1.2 volts and deliver a total of 600 milliamp-hours of charge—equivalent to 0.6 amps for one hour. For example, assume there is a circuit using this battery with a component that uses 0.1 amps per hour (100mH), then the battery could supply power for approximately 6 hours.

Larger power sources, such as wall outlets, supply much higher power. They typically output 120V AC at 15 or 20 amps. Typically, larger appliances (e.g., dryer, welder) require special outlets that supply 220 VAC. Many electronics devices (e.g., TV, computers, monitors) cannot use AC power directly. These systems use external or internal power supplies that convert the AC to DC levels appropriate for the system.

For example, a computer power adapter might convert the 120V AC / 15A input into 12V / 2A DC output. Besides their necessity for device safety, these adapters also ensure constant and reliable power. Without proper voltage and current regulation, sensitive electronics could malfunction or be damaged.

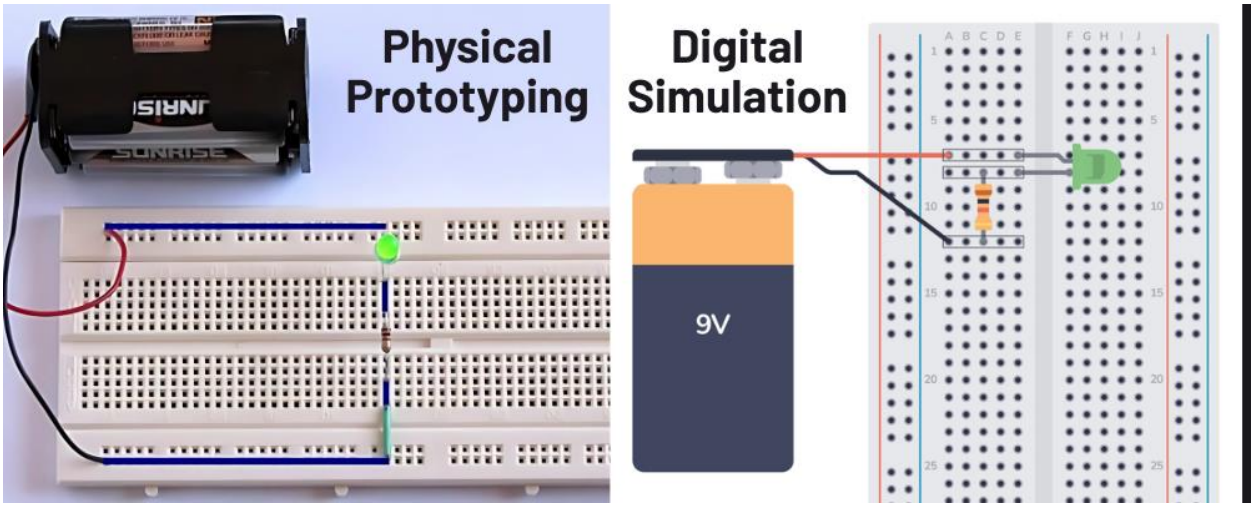
Lecture 1.4: Prototyping and Measurement Tools

Intro to Prototyping

Before committing to a permanent electronic design, engineers and technicians often begin with prototypes. Prototyping allows you to explore a design, test ideas, and identify problems early (before investing time and money into a final product). Whether the goal is to blink an LED or control a robotic arm, every design starts as a circuit that must perform a task. From there, you can determine the required components, calculate appropriate values, measure performance, and prototype the system (or parts of the system) before developing the production system.

There are two main environments that are used to prototype circuits:

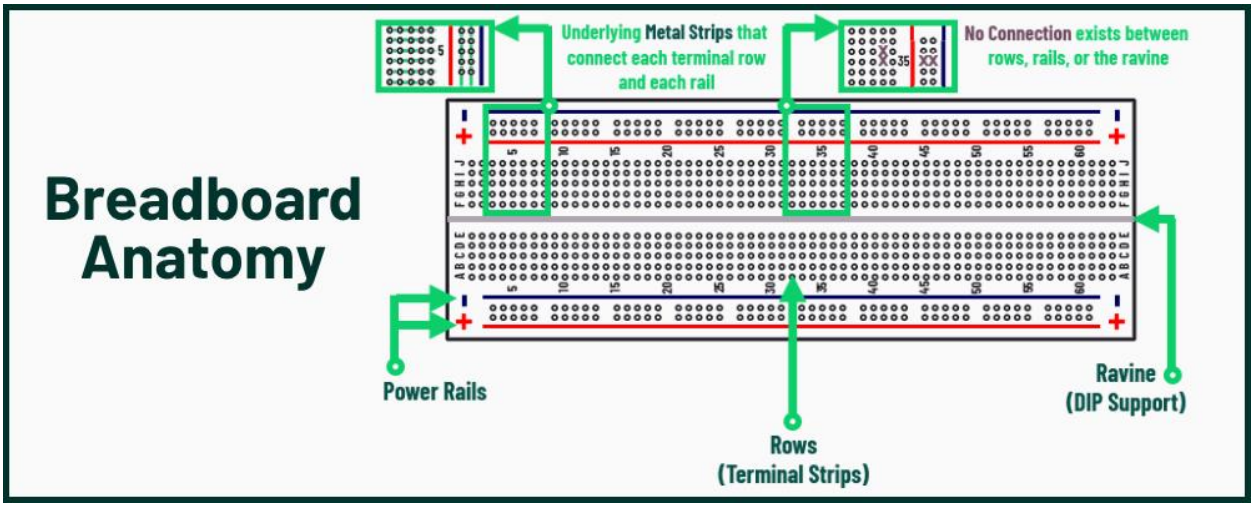
- Physical prototyping: using breadboards, multimeters, and microcontrollers to quickly build and test circuits by hand.
- Digital simulation: using circuit simulation software (like TinkerCAD or Fritzing) to model circuit behavior, experiment safely, and even write embedded code.



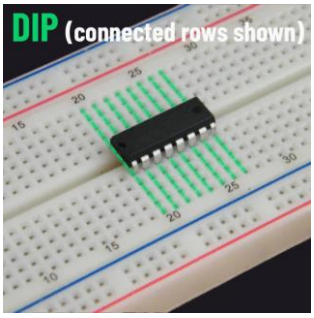
Each tool, physical or digital, serves a specific purpose. Breadboards allow rapid assembly without soldering. Multimeters help verify voltage and current. Simulators allow engineers to test circuit behavior without damaging physical components. Together these tools form a complete workflow to take an idea from a blank page to a working system.

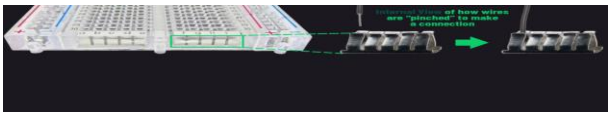
Breadboards & Rapid Prototyping

When engineers begin developing a circuit, they often use a breadboard to quickly assemble and test designs, without soldering. Breadboards are ideal for experimentation because they allow components to be inserted, removed, and rearranged easily.

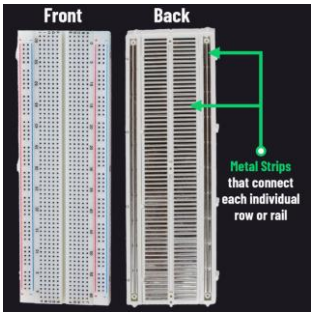


A breadboard's internal structure consists of metal strips beneath rows of holes to provide electrical continuity. Each row typically connects five holes together, allowing components in the same row to share a signal. The board is split by a center ravine, and connections do not cross it. Integrated circuits (ICs) housed in dual-in-line packages (DIPs) should be placed across this ravine to avoid shorts and provide access to all pins.



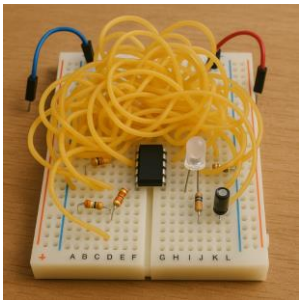


Power rails along the outer edges distribute shared access to voltage (plus) and ground (minus) across the full length of the breadboard. If you were to provide voltage to any hole along this column, it would provide voltage to the entire column.



Breadboards come in various standard sizes, such as full-sized, half-sized, or mini-sized, and may or may not include power rails. Most are compatible with common components like ICs, resistors, capacitors, transistors, LEDs, and microcontroller breakout boards.

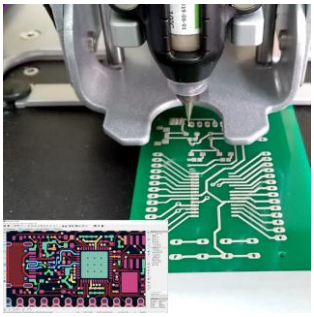
While breadboards are extremely useful, they do have limitations. For example, the connections are not permanent and can loosen over time, making them unsuitable for long-term installations or environments with vibration. They're also prone to electrical noise, making them less than optimal choices in radio frequency or high-power applications.



PCBs: From Prototypes to Products

Once a circuit has been tested and proven to work on a breadboard, it can be transferred to a more permanent platform such as a printed circuit board (PCB). A PCB is a flat board made from insulating material with conductive pathways etched or printed onto its surface. These pathways, often made of copper, connect components in a fixed layout, making PCBs more durable, compact, and reliable than breadboards, especially in robotics systems exposed to wear, vibration, and electrical noise.

Most components used during breadboarding, such as resistors, capacitors, LEDs, and even microcontrollers, can be reused on a PCB. However, because connections are permanent, building a PCB requires careful planning. PCBs are complex and are often composed of multiple layers. Software tools are used to design the board layout, define how components are connected, and generate the design files for manufacturing.



While we'll explore PCB design and fabrication more deeply in later modules, for now it's important to understand the purpose of a PCB: to provide a permanent, professional-grade version of your working prototype.

Multimeters: Measuring Circuit Behavior

The Essential Electronic Tool: A multimeter is one of the most essential tools for anyone working with electronics. It allows you to measure critical circuit values like voltage, current, resistance, and continuity, all of which help you verify and troubleshoot circuit behavior during prototyping or maintenance.

Typical Features: Multimeters come in analog and digital versions, but digital multimeters (DMMs) are the most common today. They typically feature:

- A dial or selector to choose the type of measurement
- Two probes (positive and negative) that connect to the circuit
- A digital screen that shows the measurement value

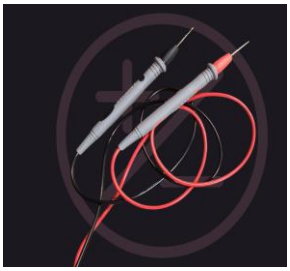
Measuring Voltage (V): Voltage is measured in parallel with a component to check how much electrical potential exists across it. The black probe gets placed on the lower-voltage side and the red probe on the higher-voltage side of the component. In robotics, you would use this type of measurement to check the power supply lines, ensuring proper voltage.

Measuring Current (A): Current is measured in series with the circuit to show how much current is flowing in the circuit. To measure the current, you must break the circuit and insert the meter in series, so that the current flows through the meter. For this reason, it can be difficult to measure current once your circuit is on a PCB. It is much easier to measure current when you are prototyping or in a simulation.

Measuring Resistance (Ω): Resistance is measured across components to verify that resistors or other devices match expected values. Resistance is measured when components are not in the circuit and/or when circuits are not powered up.

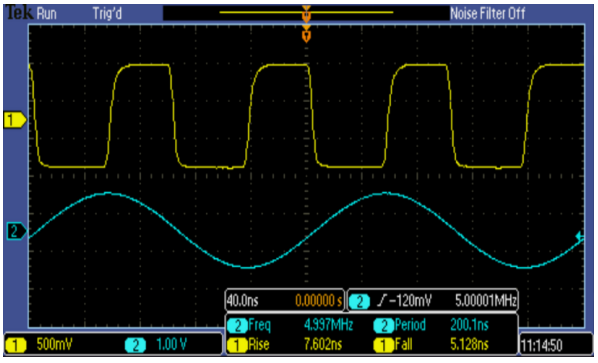
Checking for Continuity: Multimeters can do continuity checks if two points in a circuit are connected (often with a beep when the circuit is closed). Essentially a continuity check is measuring for zero resistance. Zero resistance indicates continuity. For example, in robotics, you would use a multimeter to test connections, and solder joins for continuity or open circuits.

Polarity matters: Reversing the probes won't damage most digital meters, but the reading may show a negative sign. Always follow the correct orientation to stay consistent.



Oscilloscopes: Visualizing Electrical Signals

Oscilloscopes (or o-scopes) are great for seeing how voltage changes over time, which is critical when working with signals that vary, like pulses, audio, or sensor feedback. Rather than showing a single number like a multimeter does, an oscilloscope displays a waveform: a graph that plots voltage (vertical axis) against time (horizontal axis).

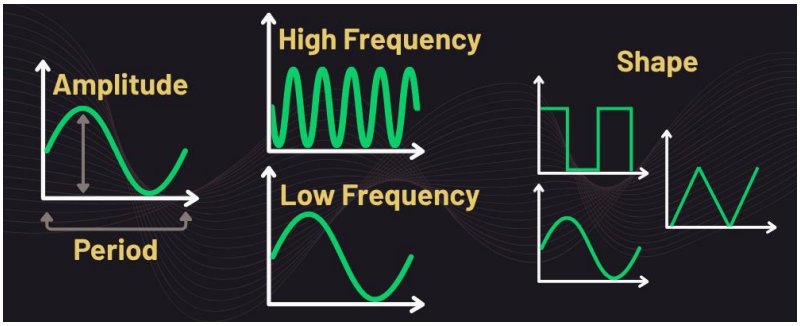


Since oscilloscopes can measure voltage over time, it allows you to monitor fluctuating signals instead of single snapshots. They can also reveal whether a signal is square, sine, or irregular. You can determine how strong the signal is (i.e., amplitude) and how fast the signal repeats, described by its period (time per cycle) or frequency (cycles per second).

Oscilloscopes help detect issues that tools (like multimeters) might miss:

- Noise – unwanted voltage fluctuations in the signal
- Glitches – fast unexpected changes in voltage

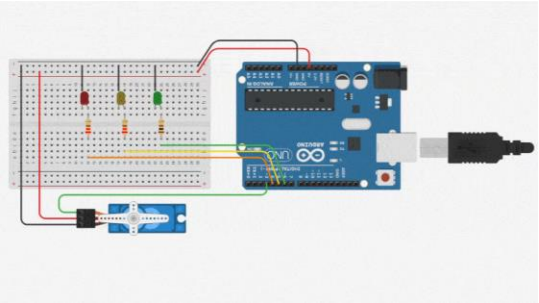
- Signal loss – when voltage drops out unexpectedly
- PWM behavior – checking how duty cycle (the proportion of "on" time) affects motor speed or LED brightness



Many robotic systems rely on signals that change rapidly. For example, motion control systems use pulse-width modulation (PWM) and analog sensors output changing voltages based on distance, temperature, or light. Microcontrollers and other digital components send digital pulses or timed signals that need to be verified for accuracy. With an oscilloscope, you can see these signals and confirm whether they're working as designed.

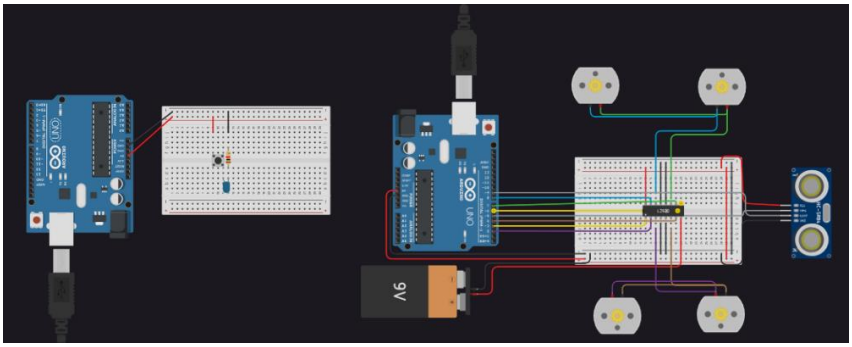
Circuit Simulators

Before building circuits with physical parts, engineers often use circuit simulators to safely test and revise their design. In this course, we'll use TinkerCAD Circuits, a browser-based tool that lets you create virtual circuits, program microcontrollers, and observe behavior in real time.



TinkerCAD allows you to place components on a virtual breadboard, wire them together, and simulate circuits. It is especially helpful during prototyping because you can safely test designs and easily reset circuits, without damaging hardware. Simulators also support embedded programming, so you can write code for Arduino boards and watch how circuits respond to microcontroller inputs and outputs.

For robotics, this means you can test how a sensor triggers a motor, or how PWM affects output, before touching real components. Simulators are not only useful for learning; they are part of the real design process.



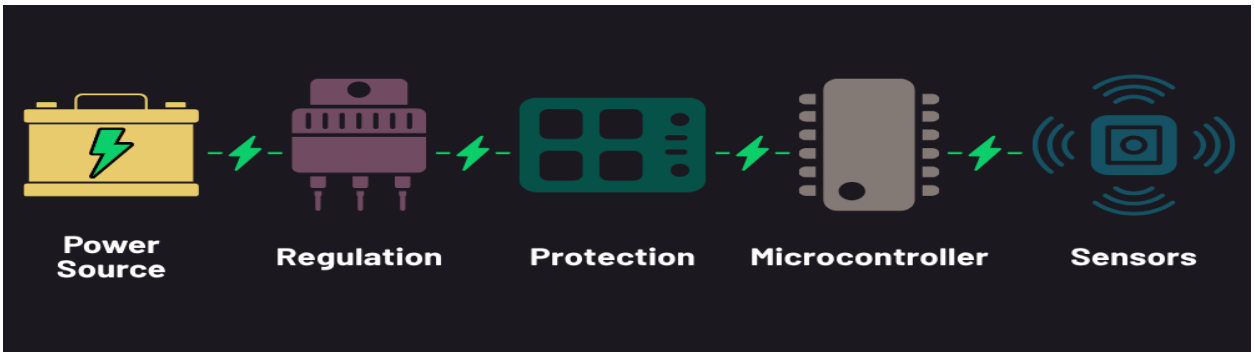
Power and Component Integration

Lecture 2.1: Power Sources and Management

Powering Robots

Power is the foundation of every robotic system. Whether a robot flies, rolls, or remains in place, it relies on a stable and consistent power supply. Designing circuits begins with understanding how to safely and efficiently deliver the right amount of power to each component. No single power supply is universal to all devices. All components, like sensors, processors, and motors, require different voltage and current levels to operate correctly, efficiently, and not burn out.

A typical power path in a robot looks like this:



The power source (e.g., battery, solar panel) feeds the voltage regulators or converters to ensure a stable output. Protection components like fuses or battery management systems (BMS) safeguard against overcurrent, short circuits, or thermal events. Once the power is properly conditioned, it is distributed throughout the system's components: sensors, computers, actuators, communications, and so forth. When selecting a power source, it is

important to consider both voltage and current requirements. Every robot component has a rated electrical operating range. Exceeding those ratings can damage the component; falling short can cause erratic or failed behavior.

Mainstream Power Sources

Common Sources

Robots operate in different environments and have different requirements based on their purpose. There are a variety of power sources to ensure efficient operation of critical functions. Here are the most common types of power sources used in robotic systems.

Rechargeable Batteries

Lithium-ion (Li-ion) and Nickel-Metal Hydride (NiMH) batteries are widely used in mobile robots due to their high energy density and reusability. They're ideal when space is limited and runtime is important, such as in drones or handheld devices.

Alkaline Batteries

These non-rechargeable batteries are typically used in low-power or short-term applications. They are common, inexpensive, and convenient, but generate waste and are not ideal for long-term or high-drain systems.

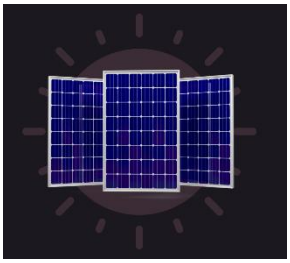
Power Supplies

Regulated DC power supplies convert wall outlet AC into stable DC voltage. They are valuable for development, prototyping, or stationary robots, especially when consistent voltage is required, but they are not practical for mobile or outdoor systems.

Generators:

Generators convert mechanical energy into electrical energy. They are less common in small-scale robotics but may be used in large, mobile, or industrial systems where long runtime is needed without battery replacement.

Solar Panels



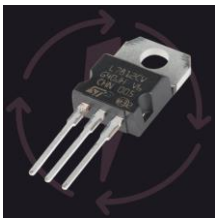
Solar Panels are great for powering long-duration outdoor autonomous systems. They convert sunlight into electricity, which is stored in rechargeable batteries and used to run the system. They're ideal for autonomous platforms with low or moderate power demands.

Hybrid Systems

Some systems combine multiple sources, such as solar panels with batteries, or batteries with capacitors, to extend runtime, improve reliability, or handle short bursts of high power. These hybrid configurations are increasingly common in field robotics and power-constrained environments.

Power Management and Protection

Portable power sources require active management to ensure reliable and safe operation. Several key components are used to provide power regulation and protection for circuits and subsystems on robotics platforms:



Voltage regulators condition the input power delivering a clean, consistent voltage, even when the input power source fluctuates above or below the required values. For example, if you're using a 7.4V Li-ion battery pack to power a 5V microcontroller, a regulator should be used to ensure the microcontroller always gets 5V, even as the battery drains.

Buck and boost converters adjust voltage up or down depending on the circuit needs. For instance, if you only have a 3.7V battery, but your motor driver needs 5V, you could implement a boost converter to raise the voltage to 5V before it reaches the motor driver. When you use a boost converter to raise input voltage, you will lower the output current because of the principle of conservation of energy. In other words, the power input must equal the power output. For example, if you have an input source of 5V at 1A, our input source power is 5 watts (5V X 1A). If we connect this input to a boost converter that provides 50% boost of the input voltage, we get 7.5V on the output. By our power equation we know current = power/output volts (5W/7.5V), meaning the output current would be 0.667A. The same conservation of energy principle applies to buck converters. Assume you have a 12V power supply, but you need to run 5V motor driver. A buck converter could lower the output voltage to 5V. For example, assume we have 12V at 1A as the input to the buck converter that reduces the output to 5V. Input power is 12W (12V X 1A) and output must also

be 12W. So, the output current = power/output volts (12W/5), meaning the output current would be 2.4A.

Battery management systems (BMS) monitor voltage, manage charging/discharging, and protect against overcurrent, deep discharge, and overheating. You should always use a BMS when working with rechargeable batteries, especially in mobile or field-deployable robots.

Fuses and protection circuits safeguard components from short circuits, surges, and reversed polarity. For example, if a motor is mis-wired or a battery is connected backwards (which could destroy your microcontroller), a fuse placed between the power source and the load would blow to disconnect power, protecting the rest of the circuit by sacrificing itself.

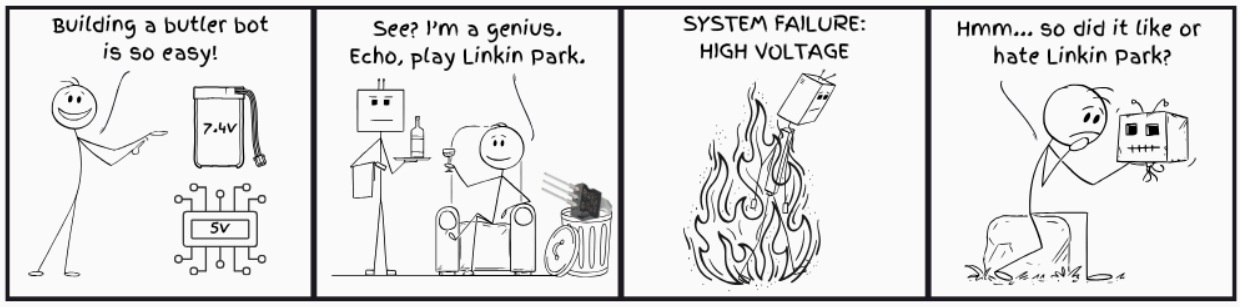
System Integration

Power management components, such as voltage regulators, converters, and protection circuits, are often used together within the same robotic system. Each one addresses a specific risk, and leaving out any one of them can lead to serious consequences.

Consider a robot powered by a 7.4V Li-ion battery connected directly to a 5V microcontroller without a voltage regulator. The board may power on briefly, but it will likely suffer irreversible damage from overvoltage.

Or imagine a robot that is powered correctly but left with no fuse to protect its microcontroller. Then, a small wiring mistake leads to a surge, and with no fuse present to cut the current, the microcontroller overheats causing permanent damage to the board.

These aren't rare mistakes; they're common oversights when treated as an afterthought. Power management and distribution is a fundamental part of system design. The right combination of components ensures that your system can recover from faults, adapt to changing conditions, and operate reliably over time.



Alternative Energy

While batteries and power management systems are common in robotics, there are scenarios, such as remote deployments or energy-hungry systems, where batteries alone aren't sufficient. In these cases, alternative energy sources can either supplement or replace conventional power systems.

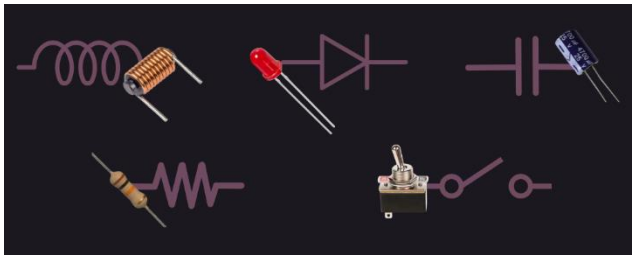
Fuel cells convert chemical energy (typically from hydrogen and oxygen) into electricity. They offer a high energy density and clean power solution that's well suited for long-duration or high-load robots.

Kinetic and thermoelectric harvesting systems generate power from movement or heat. Although their output is relatively low, they are useful for wearables or robots operating near engines or heaters.

These alternative energy sources come with unique power management requirements. For example, fuel cells would use buck or boost converters to regulate voltage and are often paired with supercapacitors to handle peak loads. Kinetic harvesters use AC-DC converters with supercapacitors to store and release energy in short bursts when needed. Thermoelectric generators (TEGs) rely on low-voltage boost converters to collect useable power from small heat gradients, including specialized chips that can operate with inputs as low as 20 mV.

Lecture 2.2: Passive and Active Components

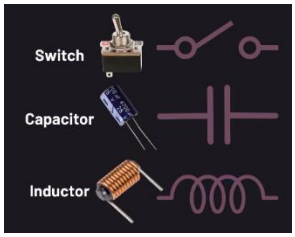
Components and Their Roles in Robotics



Electronic components are broadly categorized as passive or active. Together, these two types of components form the foundation of all electronic circuits, with passive elements shaping signal behavior and active elements enabling control, amplification, and logic.

Passive Components

Non-Powered



Passive components are essential for stabilizing, protecting, and shaping electrical signals in robotic systems. These components don't require power to function, nor do they amplify or control signals. Instead, they manage how electricity flows: limiting current, storing energy, filtering noise, and stabilizing voltage across the system.

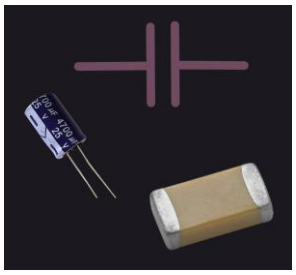
Controlling Current and Voltage:

Resistors limit current and set baseline voltages. When paired with capacitors, they can also debounce mechanical inputs, smoothing out noisy button presses or switch signals.

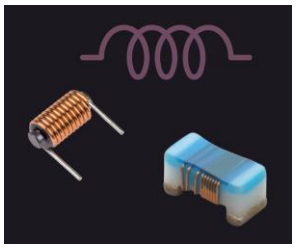
Voltage dividers, made from two resistors, scale down voltages. This is helpful for safely reading analog sensor data and protecting microcontroller inputs.

Pull-up and pull-down resistors force digital inputs, known as logic gates (HIGH or LOW), preventing unstable or "floating" behavior.

Filtering and Storing Energy (Part 1 and 2):

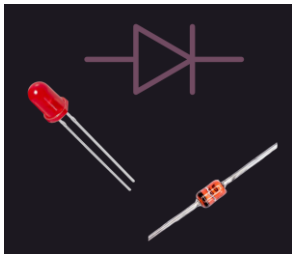


Capacitors are used to filter noise and stabilize fluctuating voltages, especially when placed near sensors and microcontroller power inputs. They absorb sudden voltage spikes from motors or switching circuits. Decoupling capacitors help keep voltage steady during sudden load changes by decoupling voltage sources from components



Inductors store energy magnetically and are commonly used in buck and boost converter circuits to manage power delivery. Inductors are often used with capacitors to suppress high-frequency noise and smooth current in power supply circuits, including those with motor drivers or switching regulators.

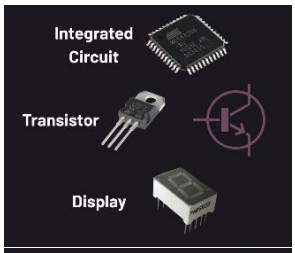
Slide 3: Directing Flow:



Diodes ensure that current flows in only one direction. They protect circuits against reverse polarity and voltage spikes, especially near motors or switching devices. Switches offer manual or mechanical control of current flow, acting as simple on/off mechanisms for various parts of the circuit.

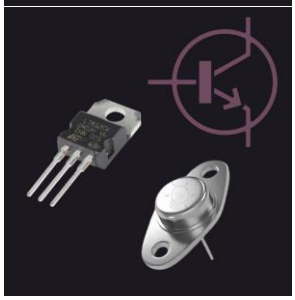
Active Components

What are Active Components?



Active components give robotic systems their intelligence, responsiveness, and control. They can amplify, switch, and generate electrical signals. Without them, a robot would be a passive circuit with no decision-making or dynamic behavior.

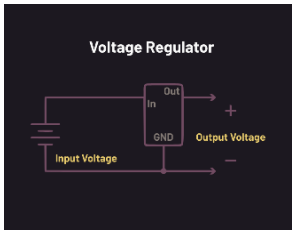
Transistors:



Transistors act as gatekeepers, allowing small control signals from a microcontroller to switch or amplify larger currents, powering motors, LEDs, or relays.

- Negative-Positive-Negative (NPN) and Positive-Negative-Positive (PNP) Bipolar Junction Transistors (BJTs) are used for low-power switching and signal amplification.
- Metal-Oxide-Semiconductor Field-Effect Transistors (MOSFETs) handle high-current loads and are commonly used in H-bridge motor control circuits for enabling bi-directional motor movement.

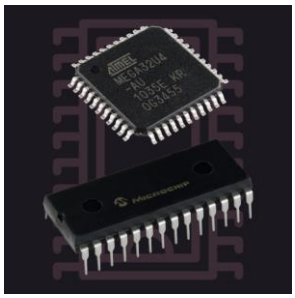
Voltage Regulators:



Voltage regulators maintain constant output voltage, despite input fluctuations

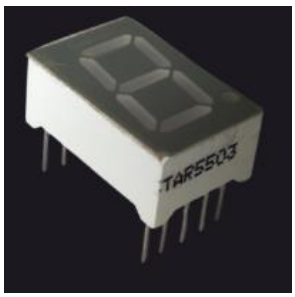
- Linear Regulators are simple and produce low electrical noise but dissipate excess energy as heat.
- Switching Regulators are more efficient, especially useful in battery-powered or high-current systems.

Integrated Circuits (ICs):



ICs combine multiple components (transistors, resistors, logic gates, etc.) into a single chip. They are used in motor drivers, logic processors, analog-to digital converters (ADCs), and timing functions like oscillators. They also reduce board space and increase reliability.

Displays:



Displays are used to communicate system status, feedback, and other outputs. They are actively controlled, often being driven by microcontrollers or display driver ICs.

- 7-Segment LEDs are simply numeric displays, often seen on digital clocks (each number has 7 segments).
- Organic-LED (OLEDs) screens can show text, graphs, or system diagnostics.

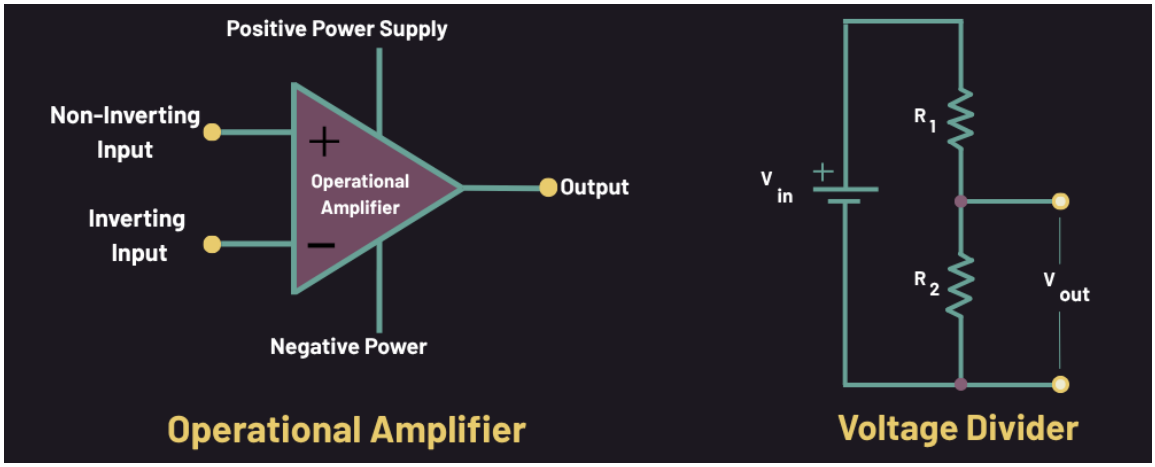
Applying components & Signal Conditioning

After you gain a foundational knowledge of how passive and active components function, it's important to further understand how they work together to condition signals, making them readable, stable, and safe for microcontrollers. Signal conditioning refers to how components modify electrical signals before they reach a processor or logic device. It is the shared responsibility between both passive and active components.

Consider two applications of signal conditioning, both passive and active respectively:

- A voltage divider, which is a pair of resistors, is used to reduce a higher voltage down to a safer level. This is useful when connecting sensors to microcontroller inputs. The formula, $V_{out} = V_{in} \times R_2 / (R_1 + R_2)$, shows how the values of two resistors determine the final voltage seen by the input.
- An operational amplifier (op-amp) is a commonly used IC that is often placed after the voltage divider to act as a buffer. The op-amp provides impedance matching, so the resistors don't interfere with the ADC's accuracy. It also improves the signal's strength to drive longer wires or additional loads without distortion.

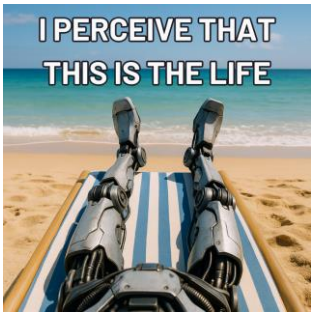
Using both circuits together is common in robotics when sensors output variable voltages and your system needs to scale voltage levels, protect inputs, and preserve signal accuracy under load.



Lecture 2.3: Sensors, Actuators, and Interfaces

From Perception to Action

Robots interact with the world by sensing their environment and responding through action. This process, perception followed by control, is foundational to robotic behavior and ties directly into the system model introduced in Module 1: Perceive > Process > Act.



Sensors give robots the ability to perceive their environment. They detect real-world conditions, such as proximity, temperature, motion, or touch, and translate them into electrical signals that microcontrollers can interpret. Actuators allow robots to perform some action based on their perception. These are devices that take commands from a controller and convert electrical energy into mechanical motion, light, sound, or other physical responses.

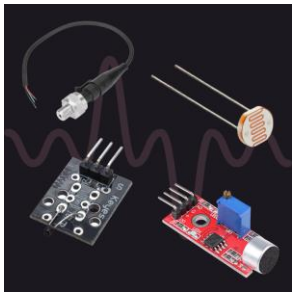
Between perception and action lies the control signal: the bridge between sensing and doing. It carries information from the microcontroller to the actuators, determining how the robot responds to its environment. Designing effective robotic systems requires careful coordination between sensor inputs, control signals, and actuator outputs.

Analog vs. Digital Sensors

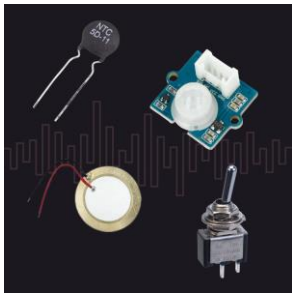
As mentioned in the previous module, sensors allow robots to perceive what's happening around them. They convert conditions like distance, movement, physical contact, or temperature into electrical signals that microcontrollers can process. When designing robotic circuits, you need to take into consideration how information is presented for your device to react to. Does it need to wait until a condition's threshold is met, or does a condition simply need to be present?

All sensors fall into one of two major categories depending on how they output their data: analog or digital. Understanding which type you are working with is key to wiring a sensor correctly and writing the right software to interpret its data.

Analog sensors produce a continuously varying voltage over time within a specific range (e.g., 0 to 5V). This output represents some physical and measurable condition (speed, temperature, etc.). Analog signals must be read by an analog-to-digital converter (ADC), either built into the microcontroller or as a separate chip.



Digital sensors provide discrete outputs, typically HIGH (i.e., the sensor sends a signal of any voltage, indicating an "on" status) or LOW (i.e., the sensor is not sending a signal, indicating an "off" status). Sensors can also send streams of digital outputs that correlate to a measurement. This makes them easier to read electrically. Examples include motion detectors, switches, temperature, humidity, and some touch sensors.



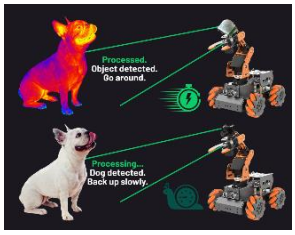
Sensor Selection & Trade-offs

Select the Sensor to Match the Task

Every sensor should support a specific decision or action a robot must make. A proximity sensor might trigger a stop command; a line sensor might guide navigation. Do not choose based on availability; select based on what needs to be detected.

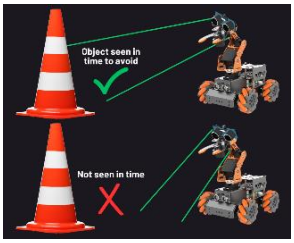
Every Sensor has Tradeoffs

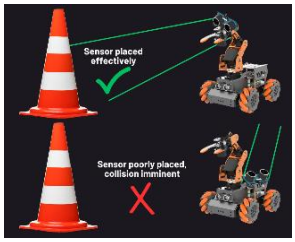
No sensor is perfect. Some sensors are fast but offer limited data (e.g., IR), while others are slower but provide richer detail (e.g., cameras). Designers must weigh speed, precision, power usage, and complexity when selecting components.



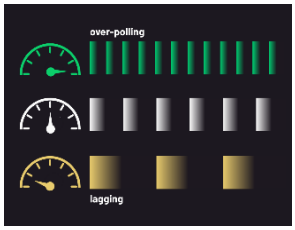
Placement, Field of View, And Range

Where and how a sensor is mounted greatly affects its performance. Poor placement can lead to false triggers, blind spots, or missed conditions. Each sensor has a specific viewing angle and range. A wide field may be affected by noise; a narrow field may miss important events.



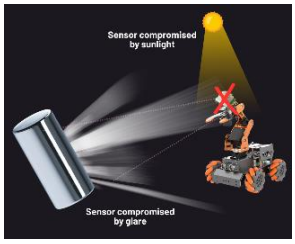


Timing and Signal Use



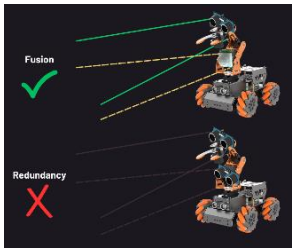
Sensors must be polled, or regularly read, at appropriate intervals. If a sensor is read too slowly, the robot may react too late. If it is read too often, it can waste processing power or amplify noise. Set polling rates based on task urgency and signal stability.

Environmental Effects



Sensors can behave unpredictably in challenging environments. Light, heat, vibration, and interference can degrade performance. A sensor may work perfectly on a test bench yet fail in operational environments. It is important to always test under realistic conditions as part of systems testing.

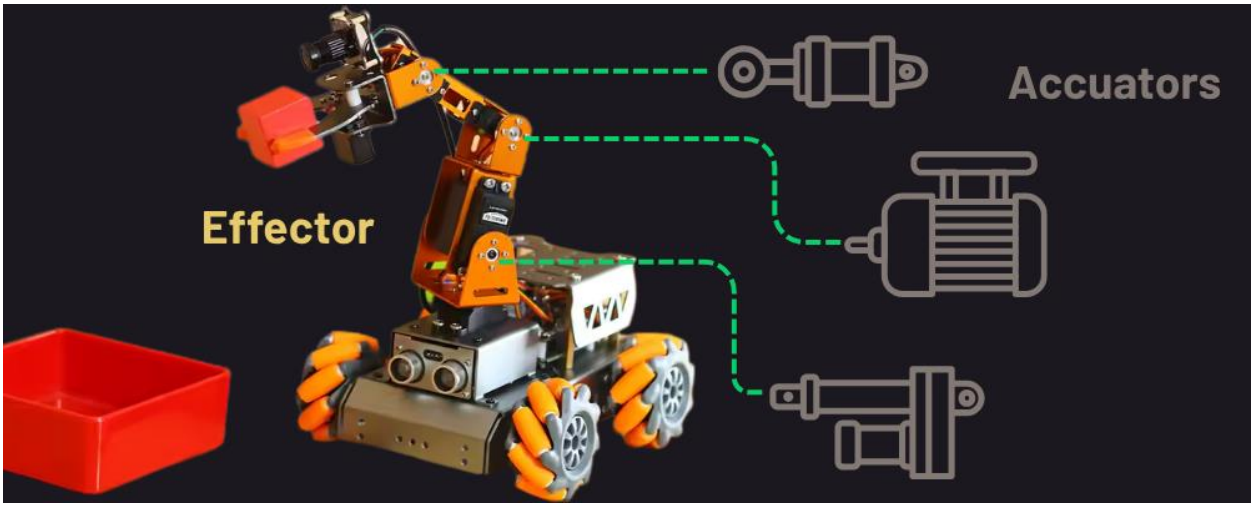
Fusion and Redundancy



In complex or safety-critical systems, robots rely on more than one sensor to detect the same condition. This helps reduce errors caused by interference, reflections, or unexpected inputs. Sensor fusion uses multiple sensors of different types to detect the same condition, such as using both IR and ultrasonic to improve obstacle detection. Redundancy means using multiple sensors of the same type to verify accuracy and add fault tolerance.

Actuators & Effectors

Once sensor data is processed and decisions are made, robots need a way to act. This is where actuators and effectors come into play; they bring life to a robot. Actuators are the components that convert electrical signals into physical movement. Effectors are the physical devices moved by the actuators that perform the robot's intended task: wheels, grippers, lights, or arms.



Most robots use some combination of DC motors, servo motors, and stepper motors:

- *DC motors* spin continuously when powered. They are fast, simple, and used for wheels or fans, but lack precise control over position.
- *Servo motors* rotate to specific angles based on a control signal. They're ideal for tasks requiring directional control, such as arm joints or sensor pivots.
- *Stepper motors* rotate in discrete steps, providing precise, repeatable motion. They're used when exact movement is critical, such as in 3D printers and Computer Numerical Control (CNC) machines.

Each actuator has trade-offs in precision, speed, torque, and control complexity. Choosing the right one depends on the task – whether the robot needs smooth driving, controlled gripping, or accurate positioning. +

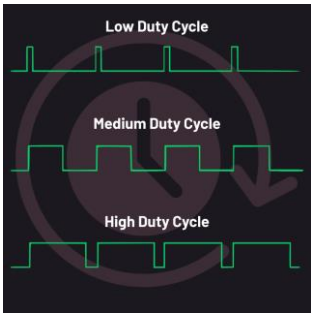
Motor Drivers and PWM Control

Microcontrollers are not designed to supply the power needed to run motors or high-load components directly. Motors typically draw more current than a microcontroller can safely provide. That's where motor drivers come in; they serve as intermediaries, or power control interfaces, allowing the microcontroller to send low-power signals that control higher-power devices.

A common type of motor driver is the H-bridge circuit, which allows motors to spin forward, in reverse, or to stop gracefully through precise switching of the power flow. Popular H-bridge modules include the L298N (for low- to medium-power DC motors) and the BTS7960 (for higher-current applications).

To control the speed and intensity, motor drivers use Pulse Width Modulation (PWM), which rapidly switches power on and off. The

ratio of on-time to total cycle time is called the duty cycle. A higher duty cycle delivers more power to the motor, increasing speed or intensity; a lower duty cycle reduces power.



Generating clean, stable control signals is important for smooth motion and reduced mechanical wear. Poor signal quality can cause jerky movement, overheating, or premature failure. A well-designed control path, such as matching signal timing, driver specs, and load conditions, helps keep robotic motion reliable, efficient, and safe.

Controllers and Digital Logic

Lecture 3.1: Microcontroller and the I/O Lifecycle

What is a Microcontroller?

In robotic electronics, a microcontroller is a decision center of the system. It reads signals from sensors, processes that information, and sends out control signals to the actuators. Unlike general-purpose computers, microcontrollers are designed to handle low-level electrical interactions directly and in real time. Their ability to interact with both digital logic and physical hardware makes them essential for embedded systems and autonomous machines. Multiple microcontrollers, with the help of more sophisticated computers, can work together to control the functionality of complex robots.

A microcontroller is a compact, self-contained computer embedded into a circuit to manage specific control tasks. In robotics, it performs three essential roles:

- Decision-making: determining how the robot should respond to sensor input
- Data processing: converting analog or digital signals into usable information
- Signal handling: generating output signals to drive motors, lights, or relays

Because they are built to interface directly with electrical components, microcontrollers typically don't need operating systems, external peripherals, or user interfaces. This makes them ideal for robotic systems where speed, power efficiency, and reliability are more important than computing complexity.

Common Microcontroller Platforms

Choosing the Right Board:

When building robotic systems, selecting the right microcontroller depends on the type and number of sensors, the complexity of actuator control, communication needs, and how quickly the system must respond to its environment.

Arduino:

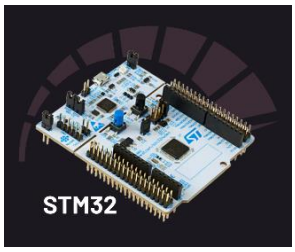
Arduino boards are beginner-friendly and commonly used in educational and hobby robotics. They offer straightforward access to GPIO pins, support analog inputs, and are great for basic timing tasks.

ESP32:

ESP32 boards include built-in Wi-Fi and Bluetooth, making them useful for wireless control or logging data remotely. They also support multitasking and have more advanced signal processing than Arduinos.

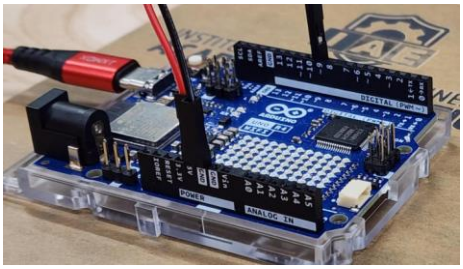
STM32:

STM32 microcontrollers are used in high-performance robotic systems or industrial robotics. They feature faster processing, precise timing, and highly configurable I/O for complex or real-time control applications. control applications.



GPIO and Signal Types

Microcontrollers connect to the physical world through GPIO pins, or General-Purpose Input/Output pins. These pins can be configured as either inputs (to receive data from sensors) or outputs (to send signals to devices like motors or LEDs). This is the most direct way for the microcontroller to manage electrical signals in robotic circuits.



Digital - Input or Output

Digital signals are the simplest form of communication. A digital input is either HIGH (on, typically 3.3V or 5V) or LOW (off, 0V). These are used to detect binary conditions such as whether a button is pressed, or a switch is open. Similarly, when a pin is set as an output, it can turn devices like LEDs or motors on or off. Think of it like a light switch; there are only two positions: on or off.

Analog – Input

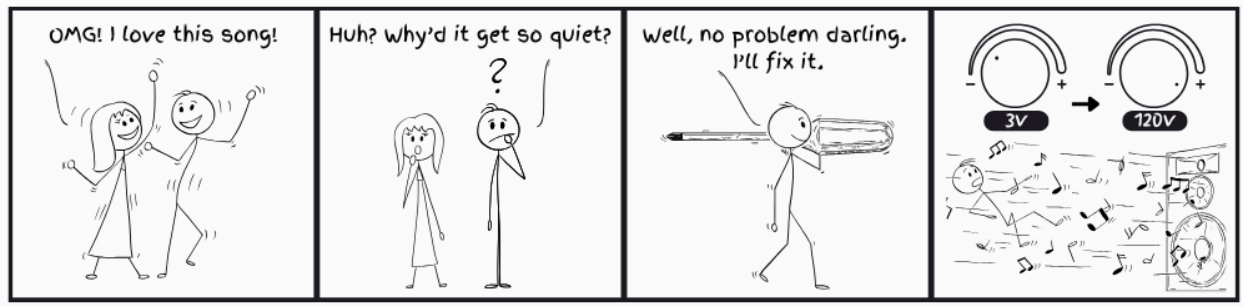
Some GPIO pins are capable of reading analog signals, which means they detect varying voltage levels rather than just high or low. This is important for sensing data like temperatures, brightness, or position. These values are interpreted through an analog-to-digital converter (ADC) inside the microcontroller.

Analog – Output

Most microcontrollers can't generate a true analog voltage, but they can simulate it using Pulse Width Modulation (PWM). PWM rapidly toggles a digital output on and off. By changing how long it stays "on" in each cycle (i.e., the duty cycle), the output appears as a smooth analog signal to motors or lights. This gives fine control over brightness, speed, or position in actuators.

Imagine flipping a light switch on and off very quickly. If it's on most of the time, the light appears bright. If it's off most of the time, the light appears dim. That's essentially how PWM works.

For example, consider potentiometers. Whether in the form of a slider or a rotary knob, they function like a dimmer switch for light or volume control. Adjusting the potentiometer changes the amount of resistance applied in the circuit, which alters the voltage at its output. This change is read by the microcontroller as an analog input.

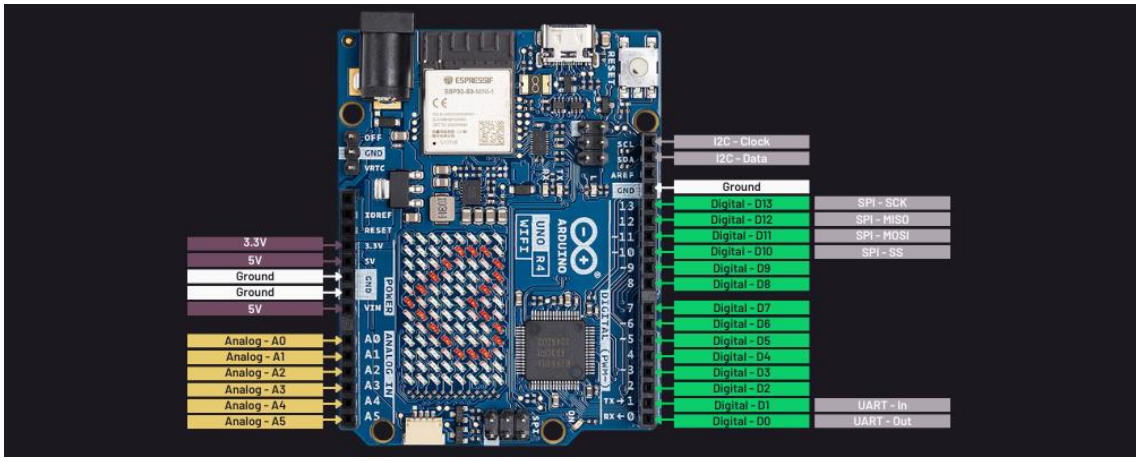


Communication Pins and Protocols

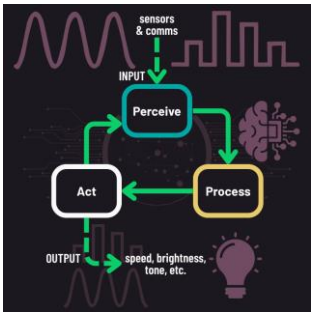
In addition to general I/O, microcontrollers use special-purpose GPIO pins for serial communication with other electronic components. These communication protocols allow the microcontroller to exchange data with displays, sensors, memory chips, or other microcontrollers. The three most common protocols are:

- Universal Asynchronous Receiver/Transmitter (UART): A simple, one-to-one communication method. It's commonly used for GPS modules, Bluetooth modules, and serial console output.
- Inter-Integrated Circuit (I2C): A shared two-wire interface (clock and data) used to connect multiple devices on the same bus – commonly used with sensors, real-time clocks, and EEPROM chips.
- Serial Peripheral Interface (SPI): A fast, four-wire protocol used for high-speed communication with devices like displays, flash memory, or ADCs.

Each protocol is built into the microcontroller and supported by dedicated pins and libraries. Understanding which protocol to use helps in designing clean, scalable robotic systems. We will go more in-depth into this topic in a later module.



The I/O Lifecycle



As previously covered, robotic circuits follow the perceive -> process -> act behavioral loop. From an electronics perspective, this means to receive input, process it, and then produce output. This section focuses on how microcontrollers handle that process through signal flow.

Inputs: Sensors send either digital (HIGH/LOW) or analog (varying voltage) signals. Digital inputs are interpreted as on/off events. Analog inputs pass through an ADC (analog-to-digital converter) to produce usable numeric values. For example, a potentiometer adjusts resistance to change voltage, which the ADC translates into a sensor reading.

Processing: Once input is received, the microcontroller runs programmed logic to evaluate conditions and determine the correct response (e.g., if-else decisions based on sensor thresholds).

Outputs: Decisions result in either digital outputs (on/off signals for motors or relays) or analog-like signals using PWM to control speed, brightness, or tone. These signals are passed to devices such as motor drivers, servos, or LEDs to carry out the action.

A well-designed I/O lifecycle ensures that every signal, from the moment it's sensed to the moment it drives motion, is timed, interpreted, and executed reliably.

Lecture 3.2: Logic and Control Timing

Signal-Based Decisions and System Timing

In robotic electronics, decisions and timing are handled through circuits that interpret electrical signals. Microcontrollers use digital logic to make decisions and timing circuits to ensure those decisions happen precisely when needed. This lecture explores how robots "think" using logic gates, how they "remember" previous inputs using flip-flops, and how timing circuits ensure signals happen in sync.

These topics may seem abstract at first, but they form the foundation of how microcontrollers process information from multiple sensors and control actuators smoothly and predictably. Whether it's stopping when both bumpers are hit or sending a signal at just the right speed, logic and timing keep robotic systems coordinated and functioning smoothly.

Basic Logic Gates

In robotics, decision-making is often handled through circuits that process simple electrical signals from sensors. Should the robot stop? Turn? Activate a motor? These decisions are made using logic gates, simple digital (on/off) circuits that follow set rules to produce a specific output based on one or more input signals.

Logic gates are the foundation of how microcontrollers interpret sensor data and control actuators. They operate on digital signals: on or off, HIGH or LOW, 1 or 0, or simply True or False. Each type of gate follows a specific rule:

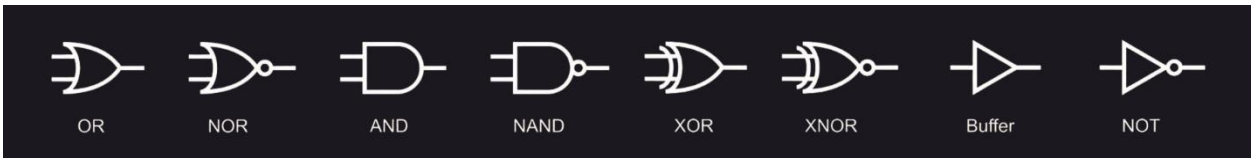
- AND: Output is True only when *both* inputs are True.
- OR: Output is True if *either or both* input is True.
- NOT: Inverts the input (True becomes False, and vice versa).
- XOR (exclusive OR): Output is True if *only one* input is True, but not both.

For example, imagine a robot with two bumper sensors: one on the front and one on the back. If only one of the bumpers is pressed, the robot may react by turning or reversing direction. If both bumpers are pressed at the same time, the robot should completely stop to avoid further damage. This behavior can be implemented with the AND gate, to send a signal to stop motion as soon as both sensors are activated.

Logic Gates in Action

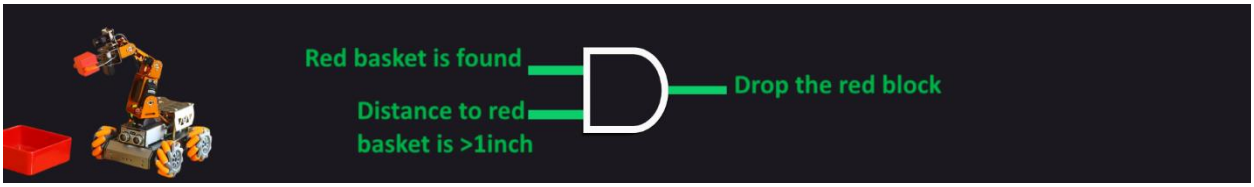
Symbols and Circuits:

Logic gates are represented by simple symbols in circuit diagrams and can be implemented in physical or simulated circuits. When combined with sensors and actuators, they form the basic building blocks of robotic decisions.



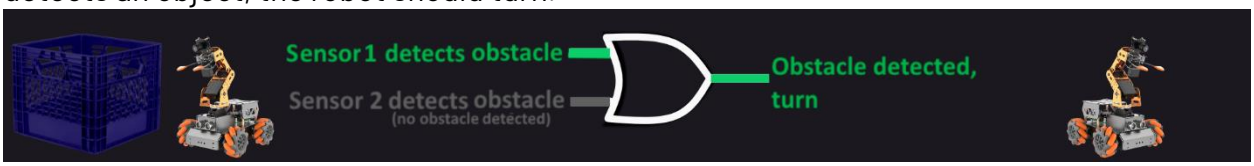
AND Gate Example:

The output signal from the AND gate is HIGH/on only when both the input signals are HIGH/on. All other input signal combinations result in a LOW/off output. This logic could be utilized in a robot arm that can only drop a red block when 1) the red basket has been located and 2) the robot is in close-enough proximity to the red basket to gently place the red block inside it.



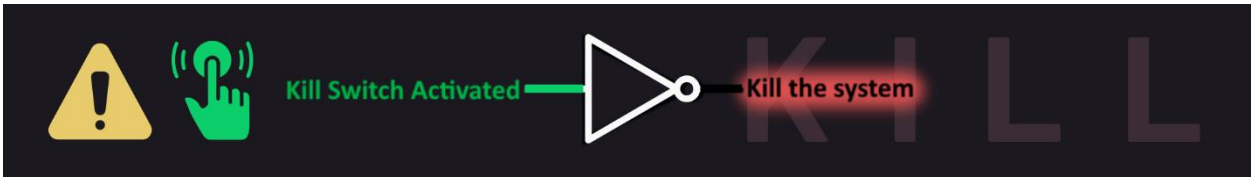
OR Gate Example:

The output signal from the OR gate is HIGH/on if either or both input signal is HIGH/on. Consider a robot that uses two infrared sensors to detect obstacles. If either sensor detects an object, the robot should turn.



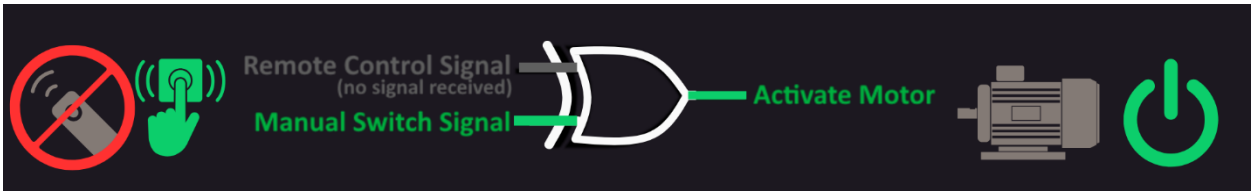
NOT:

The output signal from the NOT gate is the opposite of its input. You'll find this kind of signal inversion in a system that remains active unless a kill switch is pressed. While the kill switch is not activated, the output is High/on and the system runs. When the kill switch is activated, it travels through a NOT to produce the LOW/off signal to shut down the system.

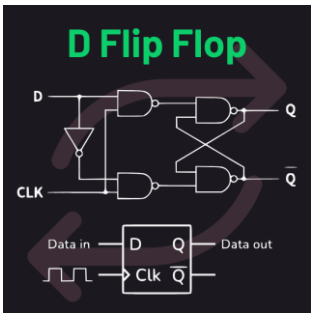


XOR:

The output signal from the XOR gate is HIGH/on only if the two inputs are opposite. If the inputs are both HIGH/ON or both LOW/OFF, the output from the XOR gate is LOW/OFF. Imagine a robot that operates a dangerous actuator that can be controlled by either a manual switch or remote control, but never at the same time. Implementing an XOR gate would ensure that it activates only when one control method is active, avoiding conflicts.



Flip-Flops



In robotics, responding to input isn't always enough; robots often need to remember what happened. Whether it's storing the last position of an actuator or remembering if a safety switch was triggered, memory is key to reliable behavior. A basic D (data) flip-flop is a circuit made up of logic gates that stores a single bit of information: either a 0 (LOW/OFF) or a 1 (HIGH/ON). This bit usually represents a system state. The flip-flop holds that value until it receives a signal to change it, allowing the microcontroller to track states even after the original input is gone.

For example, a robot with a safety switch might use a flip-flop to store whether the switch has ever been triggered. Even if the button is no longer pressed, the robot can continue to act as though the system is unsafe, until reset. Flip-flops are also used in counters, which track how many times a particular event has occurred, such as how many objects have passed a sensor. This sets up robots to carry out sequences or stop automatically after reaching a set count.

Binary & Hexadecimal

Computers operate using binary, a number system based on just two digits: 0 and 1. This simplicity aligns with the physical nature of digital electronics, where signals are either LOW (0) or HIGH (1). Every digital sensor, actuator, and communication protocol in robotics relies on binary to encode and transmit information. For example, a temperature sensor might send the binary value 00010100 to a microcontroller, which interprets it as 20°C and decides whether to activate a cooling system.

To understand binary numbers, it helps to think about how place value works in any number system. In our everyday decimal system (base-10), each digit represents a power of 10

depending on its position: the rightmost digit is the "ones" place (10^0), the next is the "tens" place (10^1), then "hundreds" (10^2), and so on. Binary works the same way, but instead of powers of 10, it uses powers of 2. Each position in a binary number represents 2 raised to a power, starting from the right. For example, the binary number 1101 can be broken down as:

$$1101$$

$$1(2^3) + 1(2^2) + 0(2^1) + 1(2^0)$$

$$1 \times 2^3 = 8$$

$$1 \times 2^2 = 4$$

$$0 \times 2^1 = 0$$

$$1 \times 2^0 = 1$$

$$8 + 4 + 0 + 1 = 13$$

Adding these individual positions together gives 8 + 4 + 0 + 1 = 13 in decimal. This system allows binary to represent any number, just like decimal, but by using only 0 and 1.

Binary (Base 2)	Hexadecimal (Base 16)	Binary (Base 2)	Hexadecimal (Base 16)
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

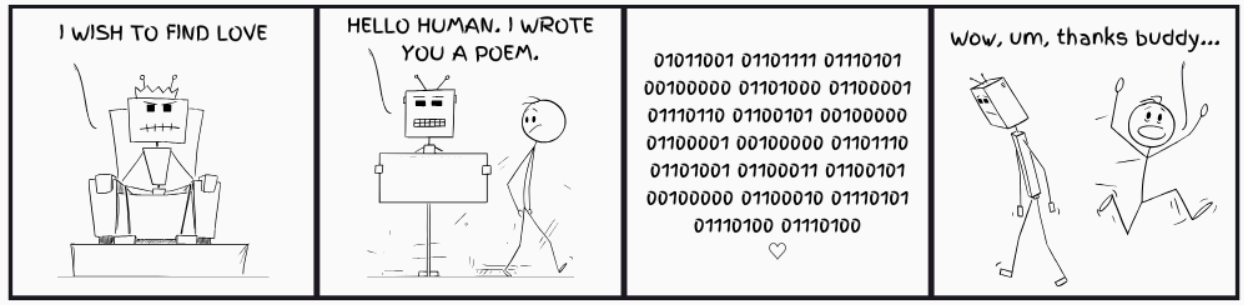
While binary is ideal for machines, it is not so friendly for engineers. Long strings of 1s and 0s are difficult to read and prone to human error. To make binary data more manageable, engineers often use hexadecimal (or hex), a base-16 number system. Hex uses sixteen symbols, 0 through 9 and A through F, where A represents 10, B is 11, and so on up to F, which is 15. Because one hex digit corresponds exactly to four binary digits, hex provides a compact and readable way to represent binary values. For instance, the binary number 11010111 becomes D7 in hex, making it easier to interpret and debug.

Understanding how to convert between binary, decimal, and hexadecimal is a useful skill in robotics. Decimal is the number system we use every day, but it does not translate easily to binary. Hex, on the other hand, bridges the gap. Consider the decimal number 255: in binary, it is 11111111, and in hex, it is FF. This kind of conversion is common when working with sensor data, memory addresses, or communication packets. Engineers often use hex when reading system logs or configuring hardware registers, because it is easier to identify and aligns well with how data is stored in memory.

In robotic systems, binary and hex are everywhere. Sensor readings are often transmitted in binary, configuration settings are stored in hex, and communication protocols like I^2C or SPI use hex to represent data packets. Even debugging tools and firmware updates rely on hex to display memory contents. When a robot receives a hex value like 3C, it might

represent a temperature, a command code, or a status flag. Knowing how to interpret these values helps engineers understand what the robot is doing and why.

Ultimately, understanding binary and hexadecimal are practical tools used daily in robotics engineering. They allow machines to operate efficiently and give humans a way to interact with those machines meaningfully. By learning to read and understand these number systems, engineers gain insight into how robots think, communicate, and respond to the world around them.



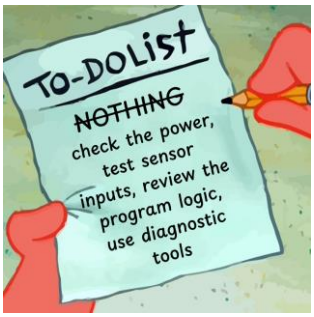
Trust, Confidence, and Safe Use

Lecture 4.1: Understanding and Debugging System Behavior

Gaining Confidence

Working with robotic systems can feel overwhelming at first. There are sensors, motors, wires, and code; each capable of causing problems that seem to appear out of nowhere. But robotic systems aren't magic boxes. With a clear strategy and some practice, it becomes possible to not only fix problems, but to understand them and even prevent them before they happen.

The first step is learning to recognize what "normal" looks like. What should the system be doing when it's working correctly? Does the motor spin when the button is pressed? Are the lights blinking in a predictable pattern? Recognizing this expected behavior helps operators quickly spot when something is off. This pattern-based intuition helps to pinpoint what to investigate and narrows down likely problem sources

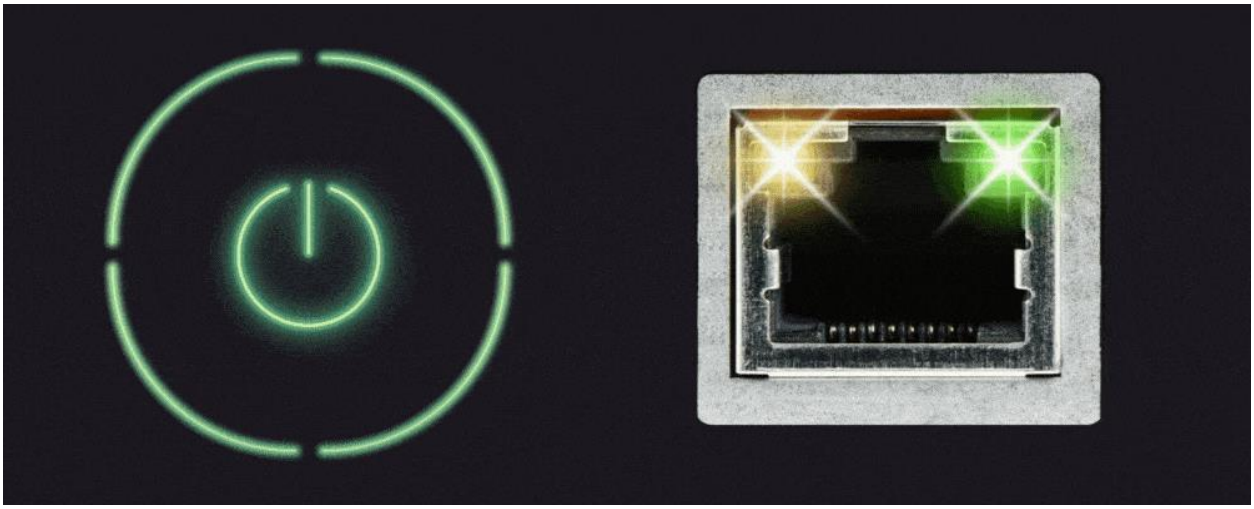


Confidence comes from both understanding and structure. Rather than guessing, experienced operators take a step-by-step approach: check the power, test sensor inputs, review the program logic, and use diagnostic tools like multimeters or serial monitors. Each step rules out possible causes and brings the system closer to a solution. When operators understand how sensors gather data, how microcontrollers make decisions, and how actuators respond, they gain not just control, but clarity. This reduces fear of the unknown and builds lasting trust in the robot's capabilities.

Recognizing Normal and Abnormal Behavior

Understanding how a robot behaves when things are working correctly is the first step in diagnosing problems. In most systems, expected behavior includes steady LED indicators, smooth actuator motion, and sensor values that change predictably. Many systems have built-in status LEDs that show power, data, or error conditions. For example, a solid green light might indicate that power is stable, while a blinking red light could signal a fault.

Recognizing these indicators helps operators quickly spot when something is off.



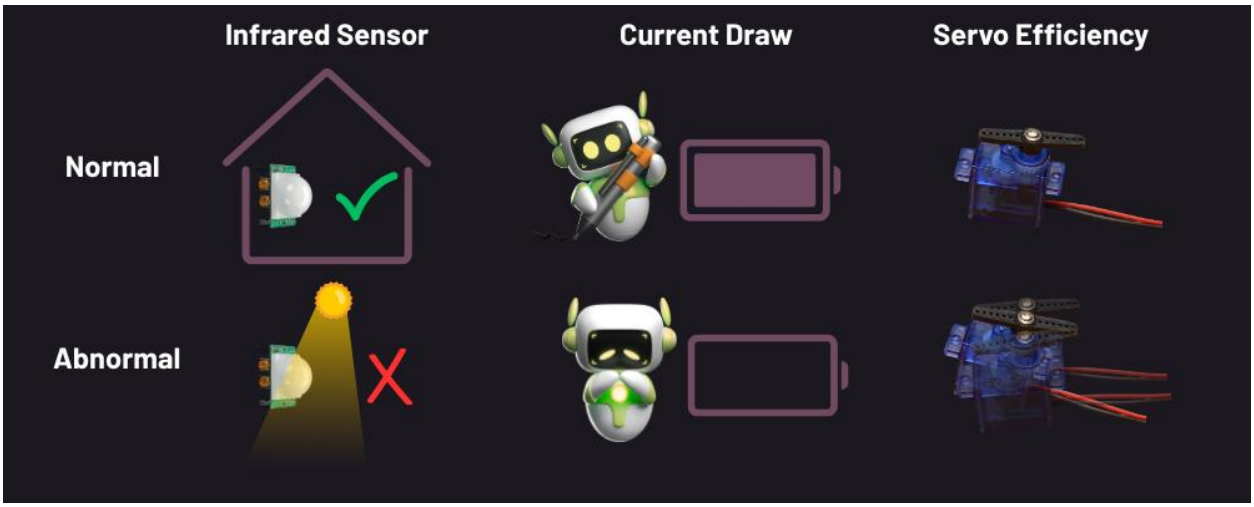
Voltage levels and sensor output also give important clues. Most microcontrollers operate within a specific voltage range. If the supply drops, such as when motors draw too much current, components may act erratically or fail altogether. A sensor reading that suddenly jumps or freezes could point to a loose wire, electrical interference, or a failing component. Knowing what's normal helps identify when a deeper issue may be present.

Building Intuition and Diagnosing Real Issues

As operators spend more time with robotic systems, they start to recognize patterns and whether a change in behavior is expected or a warning sign. For example, a motor speeding up when a load is removed is normal, but if it cuts out when another part activates, that could suggest a power supply issue. Developing this type of intuition helps operators make faster, more informed decisions.

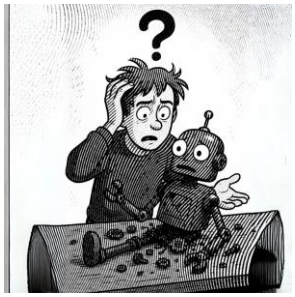
Real-world examples help build this skill:

- An infrared sensor may work fine indoors but fails in direct sunlight due to interference from ambient infrared light.
- A battery-powered robot might reset when motors engage if the battery can't supply enough current to account for the sudden current draw.
- A servo motor may jitter or buzz if the PWM signal controlling it is noisy or unstable, often due to poor grounding or timing issues.



With observation and practice, recognizing these patterns becomes second nature. This intuition gives operators the tools they need to diagnose and respond to problems efficiently and builds trust in the system's reliability.

Structured Troubleshooting Techniques



Robotic systems can fail in unexpected ways, but effective troubleshooting starts with a structured approach rather than guesswork. Whether you're working on a mobile robot, drone, sensor platform, or embedded controller, the key is to diagnose logically and systematically.

Layered Troubleshooting: Think from the Ground Up

A helpful way to structure your thinking is to treat the robot as a set of interoperating layers, each providing unique functionality to the system. Troubleshooting starts from the most basic layer, and you work through and test each until the problem can be isolated into a single layer. Here is an example framework:

1. Power and Connections (Physical Layer):

- Is the system getting power?
- Are all components of the system getting power?
- Are connectors secure and wires undamaged?
- Are power supply fuses intact and batteries charged?
- Are any components excessively warm or hot?

2. Sensors and Actuators (Component Layer):

- Are sensor values changing as expected?
- Are motors and servos moving when commanded?
- Can you feel or hear motors trying to operate?

3. Control Logic (Logic/Firmware Layer):

- Is the microcontroller running the correct program?
- Does the microcontroller pass any available diagnostic tests?
- Are you seeing debug messages or expected outputs?
- Is the system stuck in a loop or freezing?

4. Communication and Integration (System Layer):

- Are data buses (e.g., I^2C, UART, SPI, CAN) functioning properly?
- Is data flowing between modules as expected?
- Are any error indicators (blinking LEDs, beeps) present in communication components?

5. System Behavior (Application Layer):

- Is the robot performing its intended task?
- Is behavior inconsistent or failing under certain conditions?
- Is it reacting properly to inputs or environmental changes?

Divide and Conquer: Break Down the Problem

Instead of checking everything at once, a better strategy is to isolate each part of the system individually. This is a component centric approach to troubleshooting.

- Divide the system: Split your system by component — power subsystem, sensors, actuators, controller, etc.
- Swap or substitute suspect components: For example, swap in "known good" components to isolate the issue (e.g., sensor, motor, battery)
- Test components in isolation: If a motor isn't working, test it directly with a power source to rule out wiring or software issues.

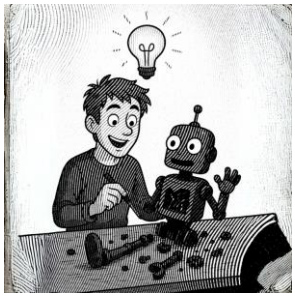
Timeline Thinking: What Changed?

Often when systems fail, the failure can be traced to something that has been changed in the system. For example, how many times have you heard or said, "it worked fine until the last update." When a system suddenly stops working, ask:

- When was it last used; When did it last work?
- What changed before it failed (e.g., new code, updates, new wiring, swapped components, routine maintenance)?
- Does the failure happen under specific conditions or at certain times (e.g., high motor load, low battery, certain commands)?

Mapping a timeline of changes to the appearance of symptoms can reveal the root cause, especially in intermittent or progressive failures.

Developing Troubleshooting Confidence



Operators who understand their systems not only have greater trust in them, but they are also empowered to fix problems on the spot. Spotting a loose ground wire or recognizing a voltage drop doesn't always require an expert—just a methodical mindset and some practice. This type of field troubleshooting and maintenance can also inform engineers designing the systems to make changes that improve future systems and/or lead to field upgrades for existing systems.

Tools for Diagnosing Hidden Problems

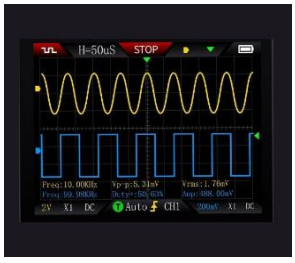
Overview:

Even with a good troubleshooting mindset, the right tools can make all the difference. Diagnostic instruments allow you to see what the robot sees, verify electrical conditions, and catch issues invisible to the naked eye.

Multimeter – Your First Responder:

A digital multimeter (DMM) is essential for checking the basics:

- Voltage: Is power reaching the board, sensor, or actuator?
- Continuity: Are connections solid, or is a wire broken internally?
- Resistance: Is a component in spec or possibly shorted/open?



Serial Console – A Window into the Code:

When debugging microcontroller-based systems, the serial console is one of the most powerful tools:

- View live sensor data.
- Catch error messages and debug output.
- Spot signs of firmware issues, like repeated resets or missing data.

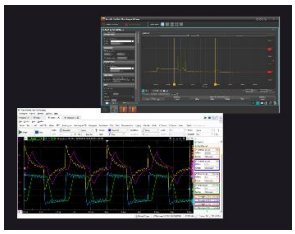
Oscilloscope – See the Signals:

An oscilloscope shows electrical signals over time and helps you:

- Visualize PWM signals, sensor outputs, and control pulses.
- Identify noise, voltage dips, or signal distortion.
- Spot intermittent issues that a multimeter might miss.

Signals from the oscilloscope can be compared with information in technical, reference manuals, or communicated to engineers to help with trouble shooting the system.

System Logs – Catch Hidden Patterns:



Logs stored by the microcontroller or host computer can reveal:

- Repeated voltage drops.
- Sensor timeouts or failures.
- Unexpected system resets or watchdog triggers.

One of the most powerful troubleshooting tools is to learn what logs your system produces, where they are, and what information they are conveying. They are key to identifying patterns or repeating anomalies that suggest cause and effect.

Supplemental Tools

- Logic analyzers: Useful for troubleshooting digital buses like I^2C or SPI.
- Logic probes indicators and test points: Provide quick visual feedback during debugging.
- Battery testers or power monitors: Identify power supply weaknesses under load.

The best troubleshooters don't just have tools—they know when and how to use them.

Communicating with Technical Teams

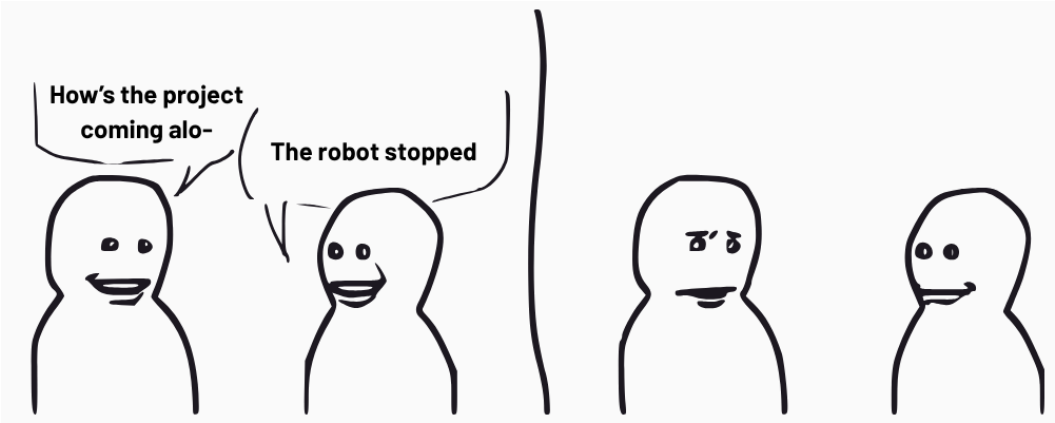
Even experienced troubleshooters encounter problems they can't solve alone. In those moments, how the issue is reported can make the difference between a fast resolution and wasted time.

Be Specific. Be Observant. Be Helpful.

Compare these two reports:

- Vague: "The robot stopped working."
- Helpful: "After turning left, the drive motor stopped, and the power LED started blinking red."

The helpful version provides context, a timeline, and observable clues—everything a support engineer needs to start solving the problem. Record any tests and results from initial troubleshooting efforts as this can be helpful to support engineers as they work through the issue.



What to Include in a Good Problem Report

Encourage your team to include:

- What was the robot doing when the issue occurred?
- What changed just before the failure? (e.g., new battery, code update, swapped components)
- What are the symptoms? (e.g., blinking LEDs, strange sounds, unresponsive sensors)
- What was already tested? (e.g., swapped sensor, reloaded firmware)
- Record and include any test information (e.g., test measurements, error messages, logs)

A concise report like this respects the time of technical support staff and increases the odds of a fast fix. Organizations may find it helpful to create standardized troubleshooting templates for their robotic systems. These templates can guide operators through common diagnostic steps, remind them of key information to record (such as observed symptoms, attempted fixes, and environmental conditions), and provide contact information for technical escalation. A well-designed template not only improves reporting quality but also accelerates resolution and knowledge sharing. Troubleshooting templates promote consistency, reduces reliance on memory, supports training and onboarding, and streamlines escalation and handoff to support engineers.

Building a Collaborative Culture

Operators don't need to be engineers, but they do need to:

- Understand the basic function of the system.
- Use the correct terminology where possible.

- Observe, document, and report clearly.

When field users and technical teams collaborate effectively, the result is faster recovery, better future designs and field modifications, and increased reliability over time.

Lecture 4.2: Building Confidence and Ethical Use

From Black Box to Glass Box

Trust in robotic systems isn't just about knowing how to fix them when they break; it's about knowing when to rely on them, and when to step in. Operators who understand how sensors influence decisions, how microcontrollers respond, and how environmental factors affect performance can anticipate failures, prevent misuse, and deploy robots with greater confidence and responsibility.

This kind of trust is earned, not assumed. It grows through repeated interactions and increasing familiarity. As operators recognize patterns and see predictable behavior, robots transform from mysterious "black boxes" into "glass boxes": transparent, explainable, and easier to trust. With each successful interaction, comprehension deepens, and confidence grows.

This creates a sustainable cycle of trust: understanding builds confidence, confidence encourages engagement, and engagement leads to more insight. With time, operators don't just trust the system; they trust their ability to use it wisely and ethically.

Recognizing Limits and Acting Responsibly

Trust doesn't mean expecting perfection. Every robotic system has limits: blind spots, power constraints, and environmental sensitivities. These limitations, often shaped by size, weight, and power (SWaP) tradeoffs, define where and how robots should be used.

A responsible operator understands these boundaries. For instance, an IR sensor may work on matte surfaces but fail to detect glossy or transparent ones. A drone might lose its GPS signal indoors, and a ground robot might struggle with slopes or uneven terrain. These aren't failures of design; they're expected limits that demand informed oversight.

Trust grows when operators don't just know what a robot can do, but also what it can't. Understanding these constraints enables better planning, timely intervention, and ethical deployment.

Knowing When Not to Trust the System

Experienced operators know that there is no room for blind faith in automation, even in the most trustworthy robotic systems. A solid understanding of electronics and system behavior helps operators weigh risks, detect issues early and prevent harm.

Even when a robot appears to be working normally, there are moments when an operator must make the judgment not to trust it. For example, a drone might fail to land autonomously due to sensor interference; in that moment, human intervention becomes critical.

Responsibility and Ethical Use

Trust, confidence, and technical skill aren't enough on their own; ethical use of robotics requires responsibility. Operators must balance what the system is capable of with how it should be used. This means asking hard questions: Is the robot suited to the task? Are there risks to people or property? Who is accountable if something goes wrong?

Responsible use includes knowing when oversight is needed, when to pause operation, and when to escalate a concern. It also means sharing what went wrong so systems can improve. Trust grows not just from technical success, but from a culture of transparency, respect, and informed judgment.

By understanding both the capabilities and the limitations of robotics systems, operators become trustworthy stewards; not just users. That's what makes automation safe, effective, and sustainable.

Module 2 Lab: Design, Build, and Analyze Circuits

PART 1: Simulate and Build Series Circuits

TinkerCAD Simulations

Robots are made up of electrical and mechanical systems that work together to achieve robot goals such as assisting humans, improving manufacturing efficiency, or achieving precise control within a broad set of applications (medical, defense, etc.). Computer-aided design (CAD) is a widely used engineering tool used to design electromechanical systems before bringing them to life in the real world. TinkerCAD is a free browser-based 3D design and electronics tool from Autodesk that makes creating digital models easy and intuitive. We will be using TinkerCAD to learn basic electronics, programming, and motor control.

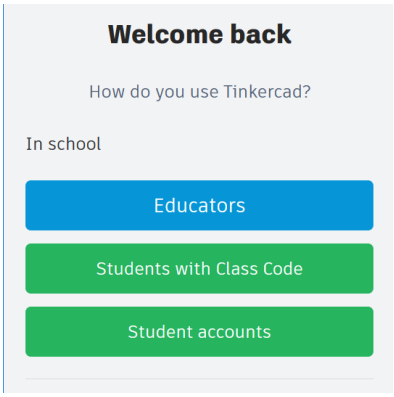
Task: Creating a TinkerCAD Account

Materials:

CrowPi2 w/ Power Cable + Mouse OR personal computer.

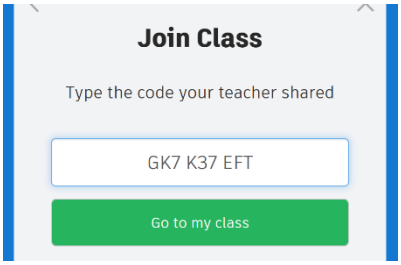
Navigate to the IRAP TinkerCAD Class

Open a web browser, navigate to tinkercad.com/login and select "Students with Class Code".



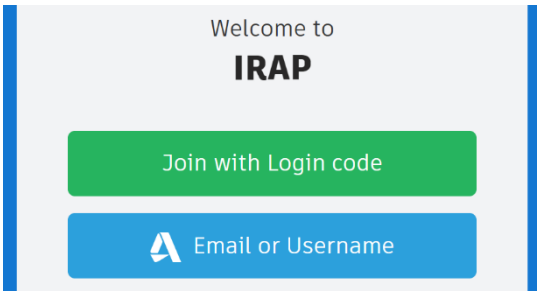
Join the IRAP TinkerCAD Class

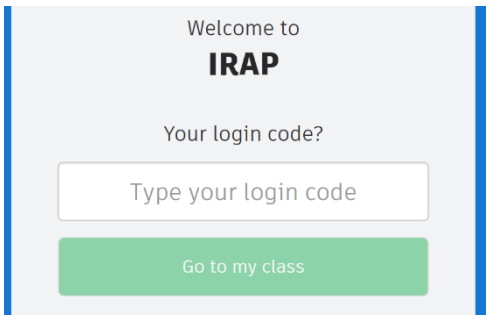
To join the class, enter GK7K37EFT into the text box.



Enter the Login Code

Click on "Join with Login code" and enter your login code, "irap###", into the text box where "###" is the IRAP kit number located on the case of the IRAP kit. TIP: *There are no spaces and all letters are lowercase.*





Simulating a Series Circuit with Two LEDs

To demonstrate the basics of electrical circuits, we will be building a series circuit that illuminates two LEDs. Before using physical components, we will be simulating the circuit with TinkerCAD. The simulation is used to ensure components such as resistors and voltage sources are sufficient for the circuitry. Simulations are a great tool to validate design ideas before spending money and time on physical components.

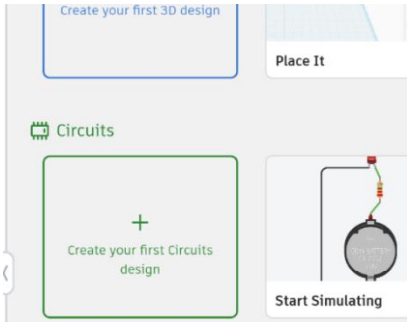
Task: Creating TinkerCAD Simulation

Materials:

CrowPi2 w/ Power Cable + Mouse OR personal computer.

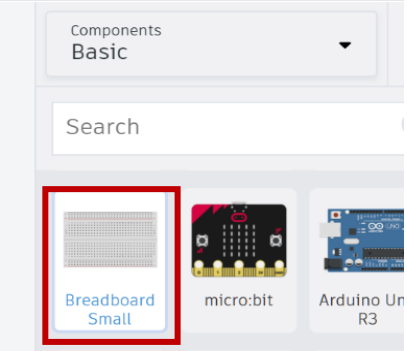
Create Circuit on TinkerCAD

Login with your TinkerCAD account and navigate to the home page. Click on the option to "Create your first circuits design" under the Circuits tab. This will create a workspace to begin building your circuit.



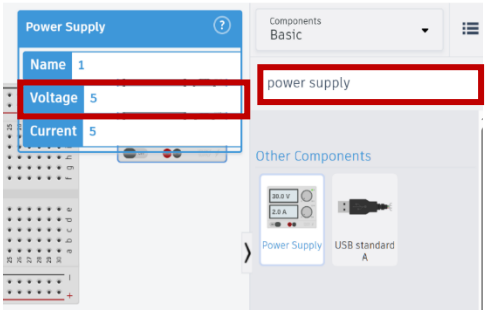
Add a Breadboard

To add components to the simulation workspace, use the tab on the right sidebar on the screen. Scroll down and click on "Breadboard Small". Once the breadboard is selected, click on anywhere in the workspace to add the breadboard to the simulation. Components can be moved around the screen by holding ctrl and dragging it with the mouse. TIP: *Scrolling in and out in your workspace will adjust the zoom level.*



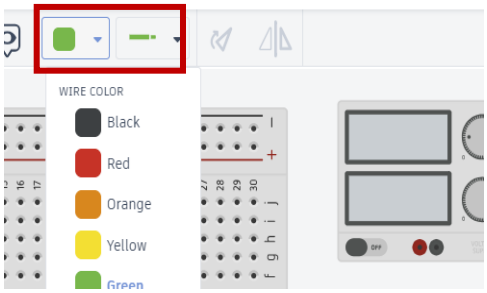
Add a Power Supply

In the Search bar, type “Power Supply” and click on it to add a power supply to the simulation workspace. Make sure the voltage is set to 5 in the power supply information bar.



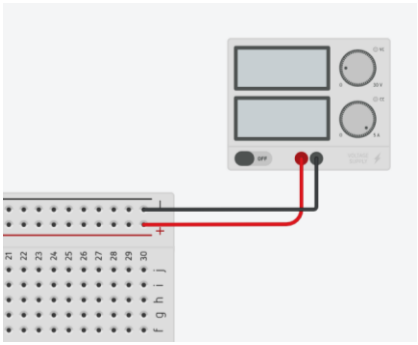
Change the Wire Color

Next, we will add a wire to connect the power supply and the breadboard. Before placing the wire, change the color to black by clicking on the colored box in the top bar. TIP: *Wire colors can be changed before or after they are placed.*



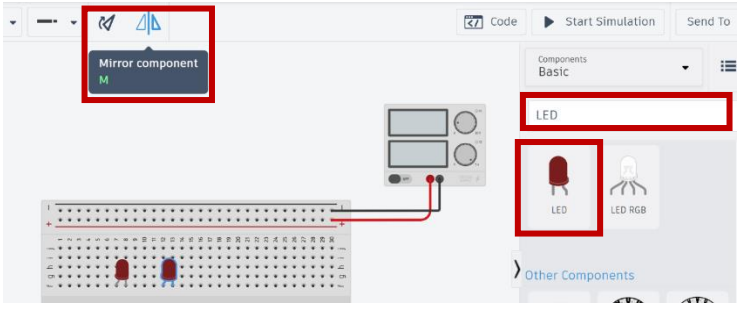
Add a Power Supply to the Breadboard

To add the negative wire from the power supply to the breadboard, click and drag from the black circle on the bottom of the power supply to the negative rail denoted with a minus symbol on the breadboard. To add the positive wire from the power supply to the breadboard use a red wire. Click and drag from the red circle of the power supply to the positive rail denoted with a “+” symbol on the breadboard. TIP*: Wires can be bent after they are placed by clicking on the wire and then clicking again at the desired bend point.”*

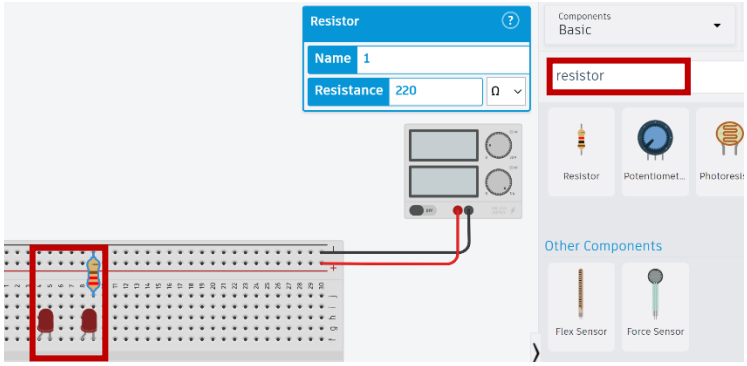


Add LEDs to the Breadboard

Add two LEDs to the breadboard. In the Search bar, type “LED” and click on the red LED. Before placing it onto the breadboard, mirror the LED by pressing “M”. Once a single LED is oriented with the *anode* (bent leg) on the left side, place it onto the breadboard. Repeat this step to add a second LED as shown in the picture provided.

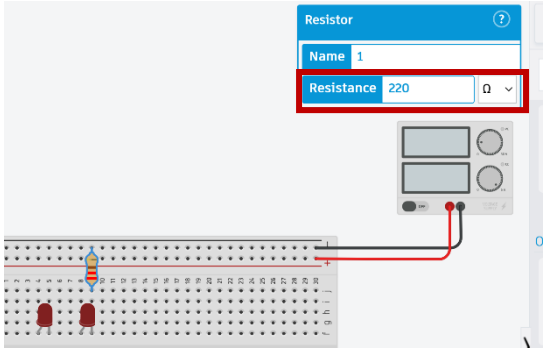


Add a Resistor to the Breadboard



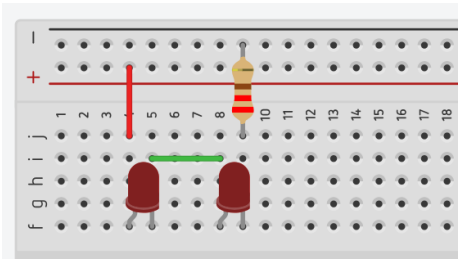
To ensure the LEDs are receiving their desired power from the power supply, a resistor is needed in the series circuit. In the Search bar, type "Resistor" and add a resistor onto the breadboard as shown in the provided image. The resistor will be connected from the *cathode* (straight leg) of the right LED to the negative rail of the breadboard. TIP: *Hovering over the component terminals with the mouse will display useful information about them. Try hovering over the LED terminals to determine the cathode and anodes.*

Configure the Resistor



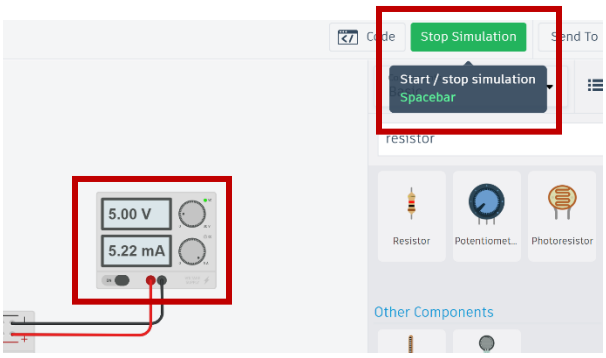
Click on the resistor placed on the breadboard to bring up the resistor menu. Enter the resistance value as 220 and then select the unit as Ω (Ohm) in the adjacent drop-down box.

Complete the Series Circuit



To complete the series circuit, add the necessary wires. Add a red wire from the positive rail of the breadboard to the *anode* of the left LED by clicking the two points on the breadboard that shall be connected. Next, add a wire to connect the *cathode* of the left LED to the *anode* of the right LED. Since the resistor connects the remaining *cathode* to the ground rail of the breadboard, it completes the series circuit.

Run the Simulation



To begin the simulation, click on the start simulation button on the top right of the screen or press the spacebar. The two LEDs should now light up. Notice the voltage and current draw displayed on the power supply. Click the stop simulation button before moving on to the next task.

Observing Voltage Drop in Series Circuit

When components are placed in series, each component experiences a drop in voltage. If too many components are placed in series together, the voltage drop will cause components to not receive enough power and thus, render the circuit inoperable. We will observe these voltage drops in simulation by placing virtual multimeters across each LED in series and reading the voltage differences.

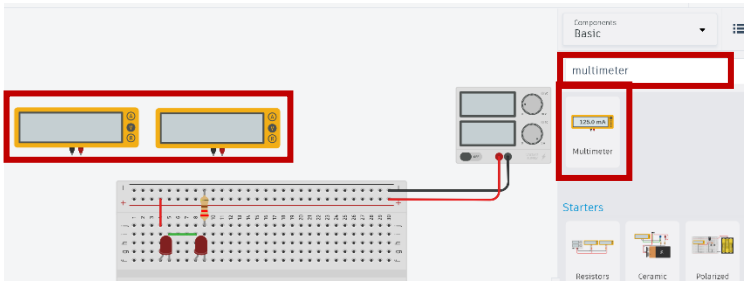
Task: Measure the Voltage of the Simulated Circuit

Materials:

CrowPi2 w/ Power Cable + Mouse OR personal computer.

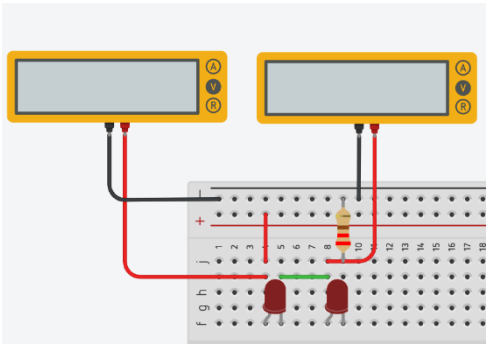
Add Multimeters to the Simulation Workspace

In the Search bar, type "Multimeter" and click on the first result. Add two multimeters to the simulation workspace.



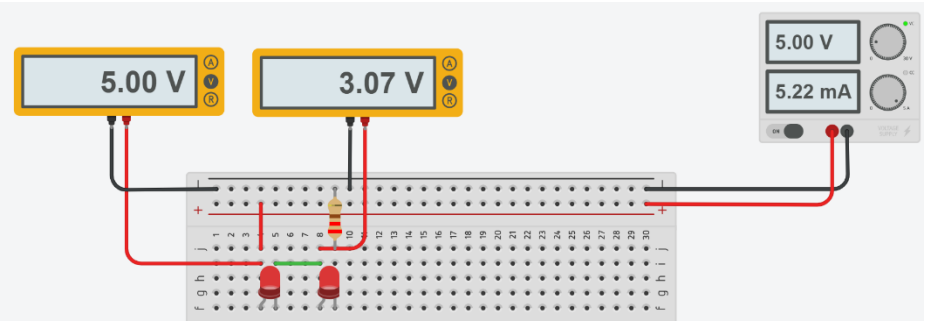
Connect the Multimeter to the Circuit

Connect the multimeters to the circuit to measure the voltage across the LEDs. Click and drag the red positive terminal on the left multimeter to the *anode* on the left LED. Click and drag the black negative terminal on the multimeter to the negative rail on the breadboard. Repeat this step for the right multimeter and LED.



Run Series Circuit Simulation

Run the simulation by pressing space bar or "Start Simulation" to view the results. The left multimeter should read 5V and the right multimeter should read about 3V. Since the LEDs are in series, a voltage-drop of about 2V is observed.



BUILDING A SERIES CIRCUIT WITH TWO LEDS

Now that the simulation is complete, we'll move on to constructing the circuit with physical components to understand how a series circuits behaves in the real world.

Task: Creating a Series LED Circuit

Materials

- 1x 2C23T Multimeter + Oscilloscope & Included Probes
- 1x 9V Battery
- 1x 9V barrel jack adapter
- 1x Breadboard
- 1x MB102 Breadboard Power Supply Module
- 1x 220 Ω Resistor
- 2x Red LEDs
- 2x Small Wires (Red and Brown)

Set-Up the Multimeter

The multimeter is battery powered and can be charged using a USB C cable provided in the package. Open the multimeter package and find the multimeter probes. The probes will be a set of black (-) and red (+) cables with a sharp metal tip at the end. Remove the plastic cap and plug the black probe to the COM terminal and the red probe to the VΩ terminal of the multimeter.

Test voltage of 9V battery

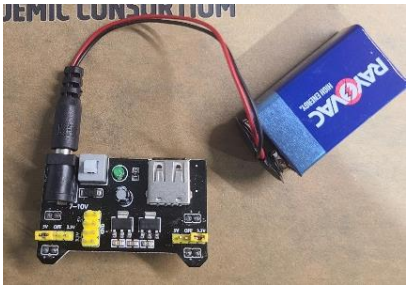
Before proceeding with building the circuit, test the voltage of the 9V battery to ensure it can be used for the circuit. Use the multimeter, navigate to the third page by using the right/left keypad buttons, then press enter to select "Multimeter". Once in the multimeter menu, it will display the text "AUTO". Test the battery by touching the black probe with the negative terminal of the battery and the red probe with the positive terminal of the battery simultaneously. The multimeter will now display a number close to 9V.

Connect 9V to barrel adapter

With the battery voltage verified, connect the 9V barrel jack adapter to the battery. TIP: *The barrel jack adapter can only be connected in one orientation.*

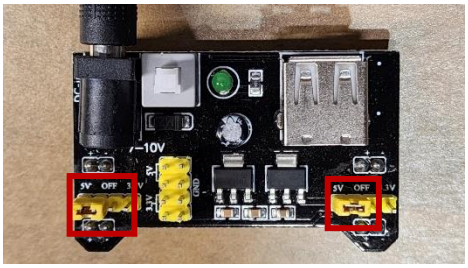
Power the MB102 Breadboard Power Supply Module

Plug the DC barrel jack into the MB102 Breadboard Power Supply Module. Test that the MB102 Breadboard Power Supply Module is working by pressing the white power button next to the DC barrel jack. A green LED will illuminate indicating power. Press the white button to turn the MB102 Breadboard Power Supply Module off before proceeding to the next step.



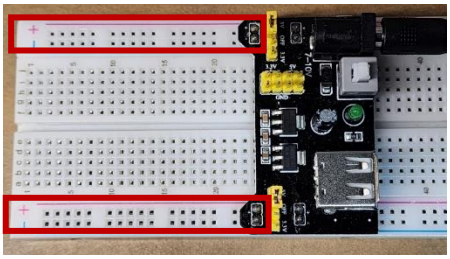
Configure the MB102 Breadboard Power Supply Module

Remove the yellow jumper headers from the MB102 Breadboard Power Supply Module and plug them into the pins labeled "5V". This will make the output of the power supply module 5V instead of 3.3V or OFF. TIP: *The MB102 Breadboard Power Supply Module allows for the left and right power rails of the breadboard to be configured independently.*



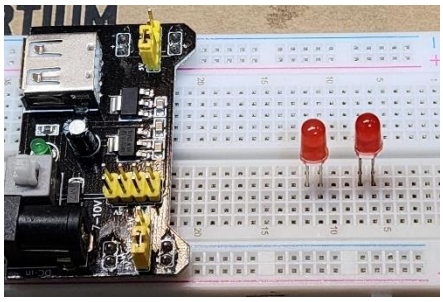
Add a Power Source to the Breadboard

Connect the MB102 Breadboard Power Supply Module to the power rails on the breadboard. This will provide 5V to the breadboard the same way that the power supply did in the TinkerCAD simulation. Ensure that the green LED is off and the + and – markings on the MB102 align with the + and – on the breadboard rails.



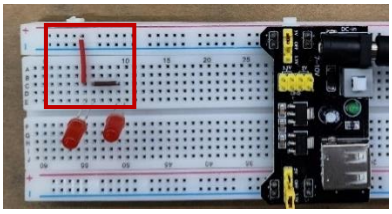
Place LEDs in the Breadboard

Like the simulation, each LED has a longer *anode* leg and a shorter *cathode* leg. Insert the LEDs into the breadboard as shown in the image provided. From left to right the LED legs should be oriented as anode, cathode, anode, cathode (long, short, long, short). TIP: *The anode*



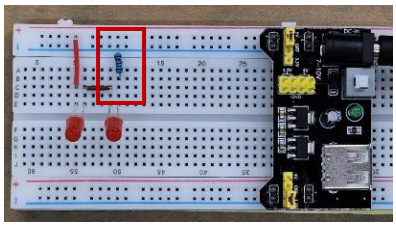
is the positive terminal of the LED and the cathode is the negative terminal.

Connect LEDs with Wires



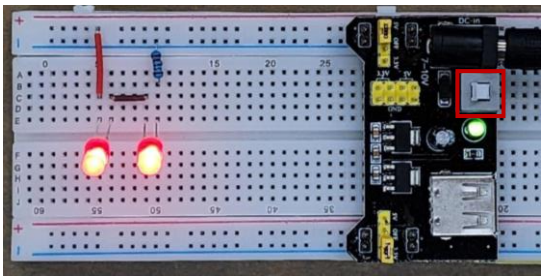
Add a red wire from the anode of the left LED to the positive rail of the breadboard. Then, add a brown wire from the cathode of the left LED to the anode of the right LED.

Add a Resistor to Complete the Series Circuit



To complete the series circuit, connect a 220-ohm resistor from the cathode of the right LED to the negative rail on the breadboard. TIP *A resistor has no polarity; it works the same no matter which way it is placed in a circuit.*

Power the Series Circuit



Carefully review the circuit using the provided image as a reference. When ready, press the white power button on the MB102 Breadboard Power Supply Module to power the series circuit. TIP *Pressing the white power button on the MB102 may cause it to lift out of the breadboard. Make sure the MB102 maintains a connection to the breadboard power rails.*

Measuring the Voltage Drop of a Series circuit

We will be using a multimeter to verify the voltage drop that we observed in simulation but in the real world. Keep in mind that the measured values in the real-world circuit may be slightly different from the simulation due to real-world factors such as component tolerances and environmental conditions.

Task: Measure the Voltage of the Physical Circuit

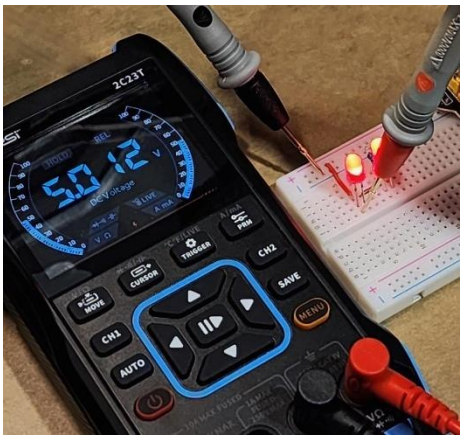
Materials

- 1x 2C23T Multimeter + Oscilloscope & Included Probes
- 1x Series Circuit from Previous Task

Set-Up Multimeter

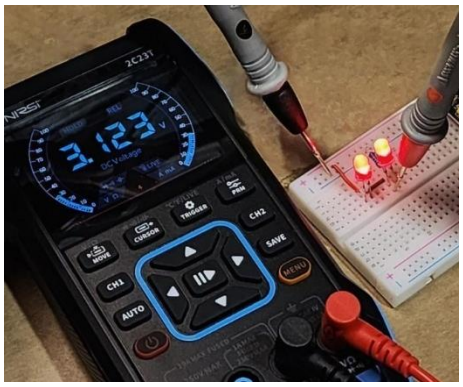
Power the multimeter, navigate to the third page by using the right/left keypad buttons, then press enter to select “Multimeter”. Once in the multimeter menu, it will display the text “AUTO”.

Measure Voltage Across the Left LED



The first measurement will measure the voltage across the left LED. Connect the red probe of the multimeter to the anode of the left LED, and the black probe to any point on the negative ground rail of the breadboard. The measurement will read around 5V. As these are real components, the value measured may differ by +/- 0.2V. TIP: *The multimeter probes can be inserted into the breadboard directly to take a measurement as shown in the provided image.*

Measure Voltage Across the Right LED



Measure the voltage of the right LED. Connect the red probe of the multimeter to the anode of the right LED, and the black probe the negative ground rail of the breadboard. Notice that the voltage dropped when compared to the left LED. LEDs requires approximately 1.8 V - 2 V to operate. The voltage of the right LED should be between 3 – 3.2 V. TIP: *If more LEDs are added in series, the voltage would continue to drop and eventually cause the LEDs to not light up.*

PART 2: Simulate and Build Parallel Circuits

Parallel Circuits

We will now simulate a parallel circuit. Previously, we observed a voltage drop across each LED in the series circuits, since all the components shared a single path. In a parallel circuit, each component will have its own path to the ground which will allow them to receive the exact same voltage. This isn't necessarily a better way to wire a circuit, it simply serves different purposes compared to a series circuit, depending on the design goals.

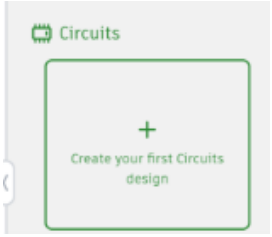
Task: Creating TinkerCAD Simulation

Materials

CrowPi2 w/ Power Cable + Mouse OR personal computer.

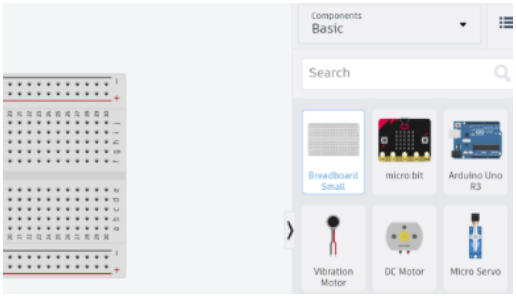
Creating Circuit on TinkerCAD

Login to TinkerCAD and navigate to the home page. Click on “Create your first circuits design” under the Circuits tab.



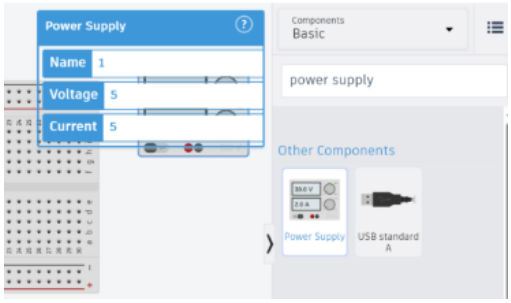
Add Breadboard

On the right side-panel, scroll down and click on “Breadboard Small” to add a breadboard to the simulation.



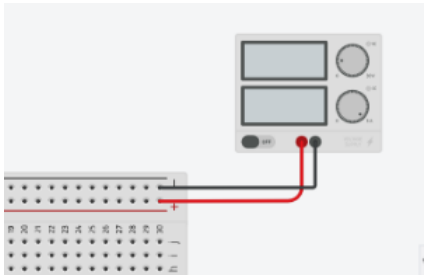
Add 5V Power Supply

In the Search bar, type “Power Supply” and click on it to add a power supply to the simulation workspace. Make sure the voltage is set to 5 in the power supply information bar.



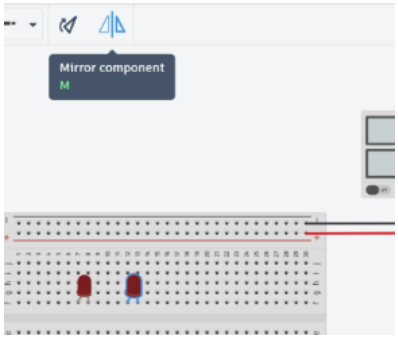
Connect the Power Supply to the Breadboard

To add the negative wire from the power supply to the breadboard, click and drag from the black circle on the bottom of the power supply to the negative rail denoted with a minus symbol on the breadboard. To add the positive wire from the power supply to the breadboard use a red wire. Click and drag from the red circle of the power supply to the positive rail denoted with a “+” symbol on the breadboard. TIP*: Use a red wire for the positive and a black wire for the negative for easy identification.*

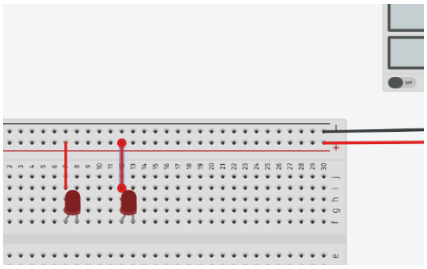


Add LEDs to the Breadboard

Add two LEDs to the breadboard. In the Search bar, type “LED” and click on the red LED. Before placing it onto the breadboard, mirror the LED by pressing “M”. Once a single LED is oriented with the *anode* (bent leg) on the left side, place it onto the breadboard. Repeat this step to add a second LED as shown in the picture provided.

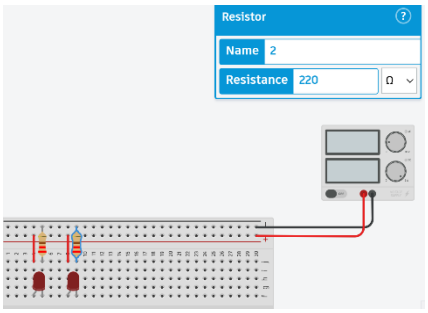


Add Wires to the Parallel Circuit



Add a red wire from the positive rail of the breadboard to the *anode* of the left LED by clicking the two points on the breadboard that shall be connected. Next, add another red wire from the positive rail of the breadboard to the *anode* of the right LED by clicking the two points that shall be connected.

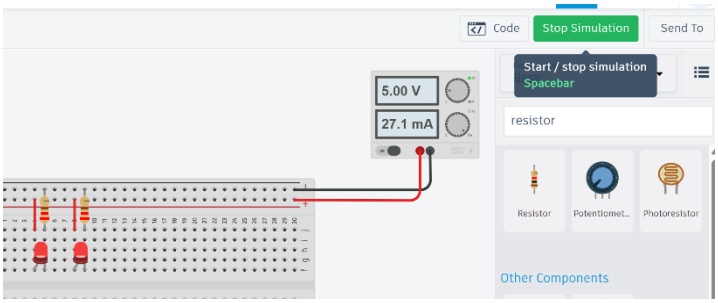
Add Resistors to the Parallel Circuit



Unlike a series circuit, a resistor is needed for each LED in parallel. In the Search bar, type "Resistor" and add a resistor onto the breadboard as shown in the provided image. The resistor will be connected from the *cathode* of each LED to the negative rail of the breadboard. Enter a resistance value of 220 Ω in the resistor information box.

TIP: *Hovering over the component terminals with the mouse will display useful information about them. Try hovering over the LED terminals to determine the cathode and anodes.*

Run the Simulation



Click the start simulation button on the top right of the screen or press the spacebar to simulate the circuit. The two LEDS should illuminate.

Observing Voltage Across Components in Parallel

In a parallel circuit, components are connected across the same two points, forming multiple paths for current to flow. One key aspect of parallel circuits is that the voltage across each component is equal to the voltage of the power source. This means every branch in a parallel circuit receives the same voltage, regardless of how many components are connected.

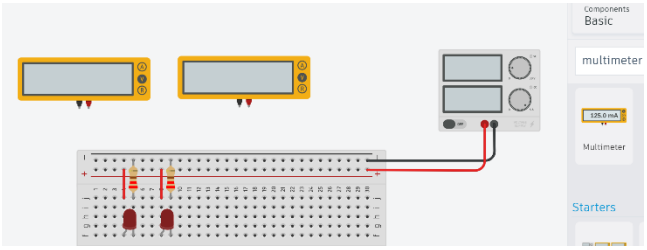
Task: Measure the Voltage of the Simulated Circuit

Materials

CrowPi2 w/ Power Cable + Mouse OR personal computer.

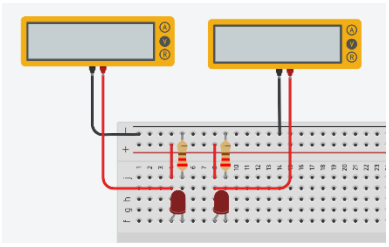
Add Multimeters to Measure Voltage

In the Search bar, type in "Multimeter", click on it, and add two multimeters to the simulation workspace.



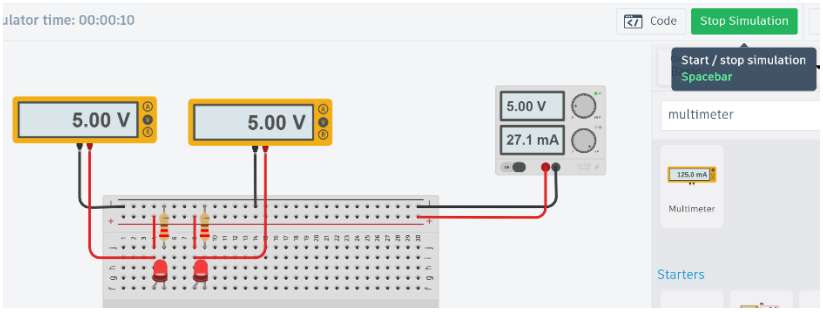
Connect the Multimeter to the Circuit

Connect the multimeters to the circuit to measure the voltage across the LEDs. Click and drag the red positive terminal on the left multimeter to the *anode* on the left LED. Click and drag the black negative terminal on the multimeter to the negative rail on the breadboard. Repeat this step for the right multimeter and LED.



Run Series Circuit Simulation

Run the simulation by pressing space bar or "Start Simulation" to view the results. The left and right multimeter should read 5V. Since the LEDs are in parallel, the voltage of the components is equal to the voltage of the power supply.



Building a Parallel Circuit with Two LEDs

Now that the simulation is complete, we'll move on to constructing the circuit with physical components to understand how a parallel circuit behaves in the real world.

Task: Creating a Parallel LED Circuit

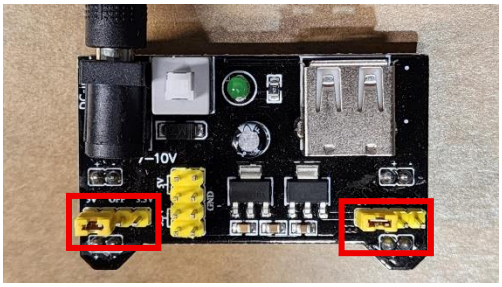
Materials

Review the following list, and set aside all components to prepare for the coming task:

- 1x 2C23T Multimeter + Oscilloscope & Included Probes
- 1x 9V Battery
- 1x 9V barrel jack adapter
- 1x Breadboard
- 1x MB102 Breadboard Power Supply Module
- 2x 220 Ω Resistor
- 2x Red LEDs

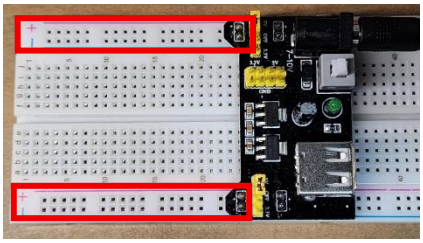
- 2x Small Wires

Configuring the MB102 Breadboard Power Supply Module



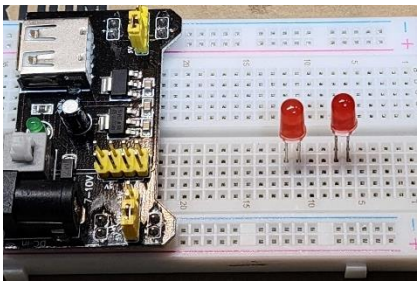
Remove the yellow jumper headers from the MB102 Breadboard Power Supply Module and plug them into the pins labeled "5V". This will make the output of the power supply module 5V instead of 3.3V or OFF. TIP: *The MB102 Breadboard Power Supply Module allows for the left and right power rails of the breadboard to be configured independently.*

Add a Power Source to the Breadboard



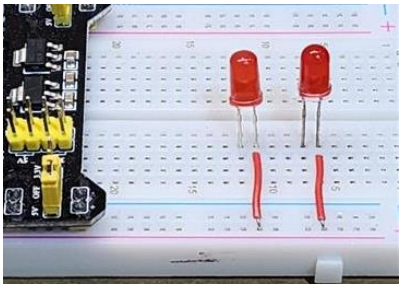
Connect the MB102 Breadboard Power Supply Module to the power rails on the breadboard. This will provide 5V to the breadboard the same way that the power supply did in the TinkerCAD simulation. Ensure that the green LED is off and the + and – markings on the MB102 align with the + and – on the breadboard rails.

Place LEDs:



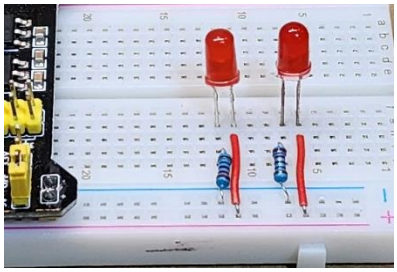
Like the simulation, each LED has a longer *anode* leg and a shorter *cathode* leg. Insert the LEDs into the breadboard as shown in the image provided. From left to right the LED legs should be oriented as anode, cathode, anode, cathode (long, short, long, short). TIP: *The anode is the positive terminal of the LED and the cathode is the negative terminal.*

Add wires to connect the parallel circuit:



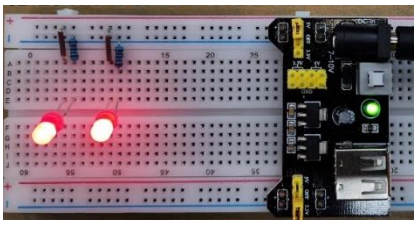
Unlike the series circuit, instead of connecting the LEDs to each other in series, they will be connected to the power rail of the breadboard individually. This is a parallel configuration. Add a wire from the anode of the left LED to the positive rail of the breadboard. Then, add a wire from the anode of the right LED to the positive rail of the breadboard.

Connect Resistors to Complete the Parallel Circuit:



To complete the series circuit, connect a 220-ohm resistor from the cathode of the left LED to the negative rail on the breadboard and another 220-ohm resistor from the cathode of the right LED to the negative rail on the breadboard. TIP *A resistor has no polarity; it works the same no matter which way it is placed in a circuit.*

Power the Parallel Circuit



Carefully review the circuit using the provided image as a reference. When ready, press the white power button on the MB102 Breadboard Power Supply Module to power the parallel circuit. TIP *Pressing the white power button on the MB102 may cause it to lift out of the breadboard. Make sure the MB102 maintains a connection to the breadboard power rails.*

Measuring the Voltage of a Parallel Circuit

We will be using a multimeter to measure the voltage of a physical circuit as we did in simulation. Keep in mind that the measured values in the real-world circuit may be slightly different from the simulation due to real-world factors such as component tolerances and environmental conditions.

Task: Measure the Voltage of the Physical Circuit

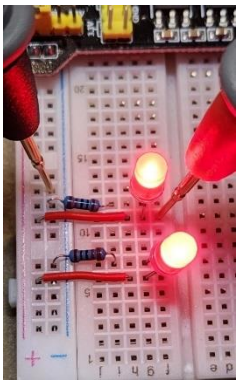
Materials

- 1x 2C23T Multimeter + Oscilloscope and Included Probes
- 1x Parallel Circuit from Previous Task

Set-up Multimeter

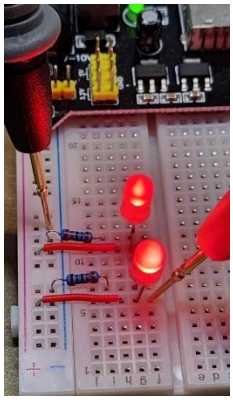
Power the multimeter, navigate to the third page by using the right/left keypad buttons, then press enter to select “Multimeter”. Once in the multimeter menu, it will display the text “AUTO”.

Measure the Voltage Across the Left LED



The first measurement will measure the voltage across the left LED. Connect the red probe of the multimeter to the anode of the left LED, and the black probe to any point on the negative ground rail of the breadboard. The measurement will read around 5V. As these are real components, the value measured may differ by +/- 0.2V. TIP: *The multimeter probes can be inserted into the breadboard directly to take a measurement as shown in the provided image*.

Measure the Voltage Across the Right LED



Now measure the voltage of the right LED. Connect the red probe of the multimeter to the anode of the right LED, and the black probe the negative ground rail of the breadboard. Notice that the voltage is also 5V and did not drop when compared to the left LED. LEDs in parallel will have the same voltage as the power source. TIP: *Remember the MB102 Breadboard Power Supply Module converts the 9V battery to 5V.*

PART 3: Controlling a Servo Motor with Pulse Width Modulation (PWM)

Motors Control Using Electrical Signals

In this lab, an Arduino will generate PWM (Pulse Width Modulation) signals to control a servo motor. An oscilloscope is an electronic instrument used to display and analyze electrical signals. It shows how voltage changes over time on a screen, with voltage on the vertical axis and time on the horizontal axis. This allows precise observation of the shape, frequency, amplitude, and timing of signals in a circuit.

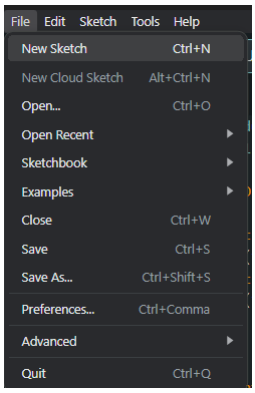
Task: Creating a Motor Control Script

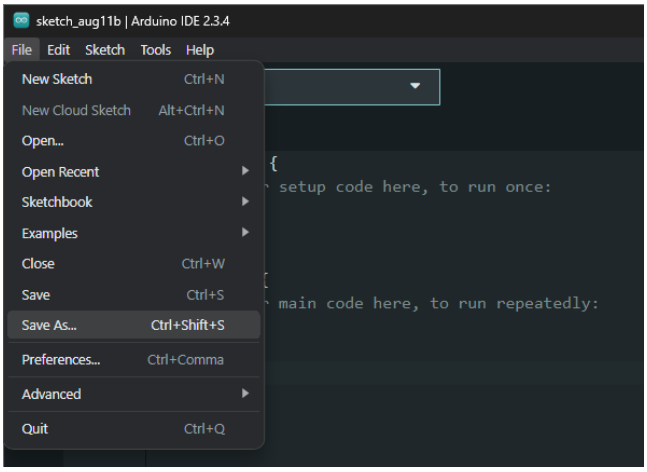
Materials

CrowPi2 w/ Power Cable + Mouse OR personal computer.

Create a New Arduino Sketch

Open the Arduino IDE and create a new file by selecting “File”, then “New Sketch”. Once a new sketch is opened, click on “File” then “Save As” and name the file “Servo_PWM”. The objective of the script will be to move a servo motor between a defined set of angles using a PWM signal.

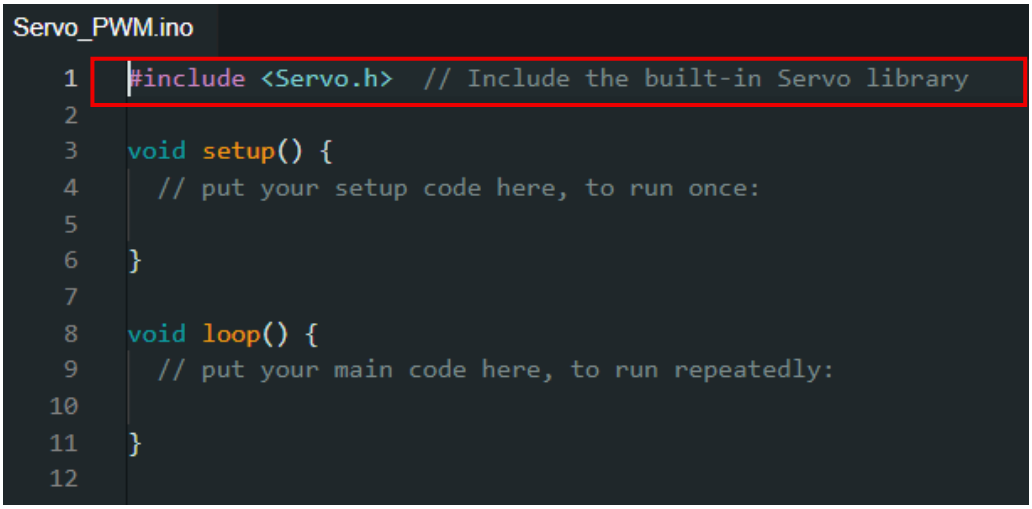




Import the Servo Library

Add the Servo library by typing “#include <Servo.h>” in line 1 of the code. This library will simplify the code needed to control the servo motors. TIP: *Libraries provide pre-written, reusable code that developers can incorporate into their projects to save time and effort.*

```
#include <Servo.h> // Include the built-in Servo library
```



Create Servo Object

Below the Servo library (#include <Servo.h>), create the servo object by typing “Servo myServo;”. TIP: *Notice the added comments in the beginning of the script. It is generally good practice to comment on the purpose of the code in the beginning to give context to other developers or yourself later. These comments do not affect the functionality of the code.*

```
Servo myServo;       // Create a servo object named "myServo"
```

Servo_PWM.ino

```
/*
  Arduino Uno - Servo Motor Control Example
  -----------------------------------------
  This program moves a servo motor to 0°, 30°, 60°, 90°, and 180° positions
  one after another, with a short pause at each position.

  Hardware needed:
    - Arduino Uno
    - Servo motor
    - Jumper wires

  Servo motor wiring:
    - Brown             -> GND (Arduino ground pin)
    - Red wire          -> 5V (Arduino 5V pin)
    - Orange            -> Digital pin 9 (signal pin)
*/

#include <Servo.h>  // Include the built-in Servo library

Servo myServo;      // Create a servo object named "myServo"

void setup() {
  // put your setup code here, to run once:


}

void loop() {
  // put your main code here, to run repeatedly:

}
```

Initialize the Servo

Inside of the setup function (void setup() {}) add the provided code to initialize the servo motor and delay time between each angle change. TIP: In Arduino programming, w*hitespace (Spaces or Tabs) does not affect the execution of code*:

```
// Attach the servo signal wire to digital pin 9
myServo.attach(9);

// Optional: Move servo to 0 degrees when program starts
myServo.write(0);

// Give the servo time to reach the position
delay(1000);
```

```
20  Servo myServo;      // Create a servo object named "myServo"
21
22  void setup() {
23    // put your setup code here, to run once:
24    // Attach the servo signal wire to digital pin 9 on the Arduino
25    myServo.attach(9);
26
27    // Optional: Move servo to 0 degrees when the program starts
28    myServo.write(0);
29
30    // Give the servo a moment to reach the position before starting
31    delay(1000);
32
33  }
34
35  void loop() {
```

Program the Main Loop

In the loop function (void loop(){}), add the following lines of code to the script. This code will tell the Arduino to move the servo to 0°, 30°, 60°, 90°, and 180° in sequence, pausing one second at each position so you can see the movement. After reaching 180°, it immediately starts again from 0° and repeats forever. This endless cycle is what makes the servo continuously move between the set angles without you needing to restart the program. TIP: *Pin 9 is a PWM enabled pin and can be used for normal GPIO use and PWM control.*

```
// Move the servo to each angle with a delay in between

myServo.write(0);     // Move to 0 degrees
delay(1000);          // Wait 1 second

myServo.write(30);    // Move to 30 degrees
delay(1000);          // Wait 1 second

myServo.write(60);    // Move to 60 degrees
delay(1000);          // Wait 1 second

myServo.write(90);    // Move to 90 degrees
delay(1000);          // Wait 1 second

myServo.write(180);   // Move to 180 degrees
delay(1000);          // Wait 1 second

// Loop starts over automatically
```

```
  // Give the servo a moment to reach the position before starting
  delay(1000);

}

void loop() {
  // put your main code here, to run repeatedly:
  // Move the servo to each angle with a delay in between

  myServo.write(0);    // Move to 0 degrees
  delay(1000);         // Wait 1 second

  myServo.write(30);   // Move to 30 degrees
  delay(1000);         // Wait 1 second

  myServo.write(60);   // Move to 60 degrees
  delay(1000);         // Wait 1 second

  myServo.write(90);   // Move to 90 degrees
  delay(1000);         // Wait 1 second

  myServo.write(180);  // Move to 180 degrees
  delay(1000);         // Wait 1 second

  // Loop starts over automatically
}
```

Revise and Save Code

The following is the finalized code (what your complete program should look like) after following the previous steps. This code will be used in the following tasks. Ensure it is saved clicking "File," then "Save" or using CTRL + S on the keyboard.

```
/*
  Arduino Uno - Servo Motor Control Example
  -----------------------------------------
  This program moves a servo motor to 0°, 30°, 60°, 90°, and
180° positions
  one after another, with a short pause at each position.

  Hardware needed:
    - Arduino Uno
    - Servo motor
    - Jumper wires
```

```
  Servo motor wiring:
    - Brown             -> GND (Arduino ground pin)
    - Red wire          -> 5V (Arduino 5V pin)
    - Orange            -> Digital pin 9 (signal pin)
*/

#include <Servo.h>  // Include the built-in Servo library

Servo myServo;      // Create a servo object named "myServo"

void setup() {
  // put your setup code here, to run once:
  // Attach the servo signal wire to digital pin 9 on the
Arduino
  myServo.attach(9);

  // Optional: Move servo to 0 degrees when the program
starts
  myServo.write(0);

  // Give the servo a moment to reach the position before
starting
  delay(1000);

}

void loop() {
  // put your main code here, to run repeatedly:
  // Move the servo to each angle with a delay in between

  myServo.write(0);    // Move to 0 degrees
  delay(1000);         // Wait 1 second

  myServo.write(30);   // Move to 30 degrees
  delay(1000);         // Wait 1 second

  myServo.write(60);   // Move to 60 degrees
  delay(1000);         // Wait 1 second

  myServo.write(90);   // Move to 90 degrees
  delay(1000);         // Wait 1 second
```

```
  myServo.write(180);  // Move to 180 degrees
  delay(1000);         // Wait 1 second

  // Loop starts over automatically
}
```

Using an Arduino in TinkerCAD

In this task, you will explore how an Arduino Uno can generate a Pulse Width Modulation (PWM) signal to control a servo motor. Using TinkerCAD's simulation environment, you will build a virtual circuit that includes an Arduino Uno, a servo motor, and an oscilloscope. The Arduino will send PWM control signals to the servo, while the oscilloscope will allow you to visualize the signal's waveform in real time. By observing how changes in the PWM signal affect the servo's position, you will gain a deeper understanding of how digital control signals are translated into precise mechanical motion.

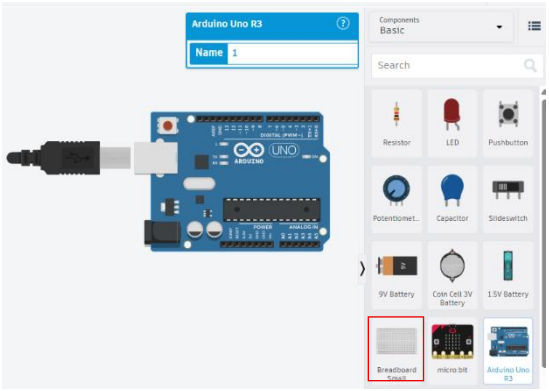
Task: Simulating a PWM Signal

Materials:

CrowPi2 w/ Power Cable + Mouse OR personal computer.

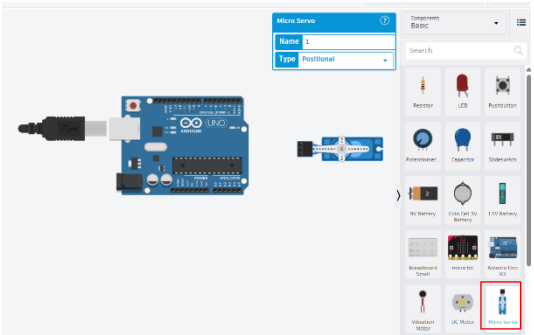
Add an Arduino

Add an "Arduino Uno R3" from the right side-panel to the simulation workspace.



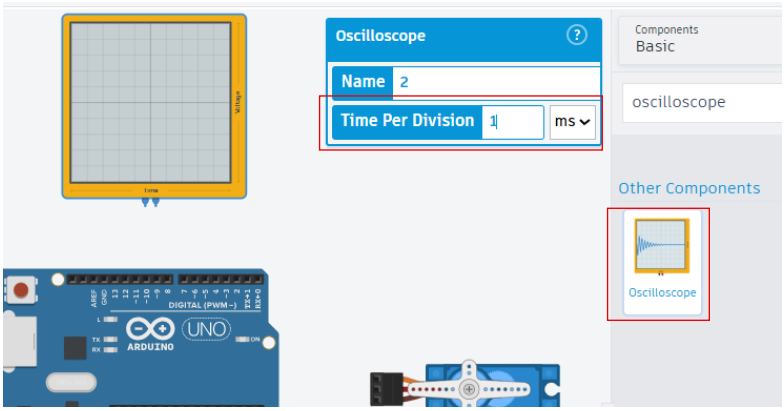
Add a Servo

Add the "Micro Servo" motor to the simulation workspace from the right side-panel. Use Shift + R to rotate the servo counterclockwise. Rotate the servo unit until it is oriented as shown in the provided image. The servo motor's input should be facing the Arduino.



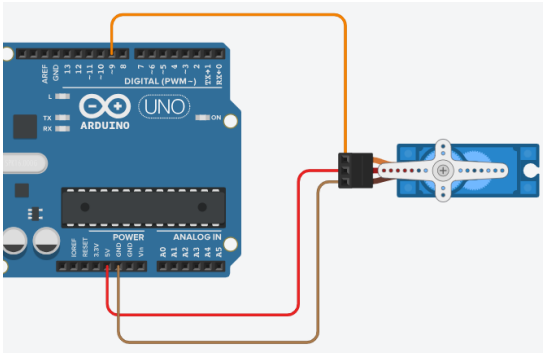
Add an Oscilloscope

In the Search bar, search: "Oscilloscope". Click on the oscilloscope to add it to the simulation. Ensure that the Time Per Division is set to 1 ms in the oscilloscope information box.



Wire the Circuit

Three wires are needed to connect the servo motor to the Arduino so it can receive control signals from the Arduino:



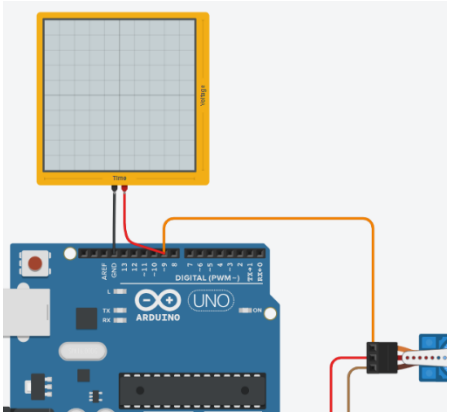
- Connect the orange signal pin of the motor to pin 9 of the Arduino.
- Connect the red Power pin of the motor to the 5V pin on the Arduino Uno.
- Connect the brown Ground pin of the motor to the GND pin of the Arduino Uno.

TIP: The *GND pin on the Arduino Uno is ground. Notice there are three ground pins available on the Arduino Uno. All the grounds are electrically connected and act as the same ground despite having multiple pins.*

Connect the Oscilloscope to the Circuit

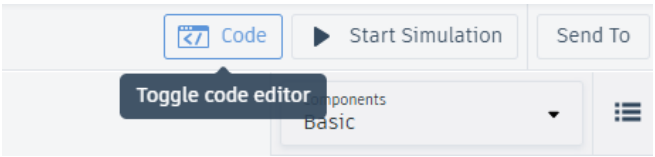
To measure the PWM output from the Arduino:

- Connect the red positive terminal of the oscilloscope to pin 9 of the Arduino.
- Connect the black negative terminal of the oscilloscope to the GND pin of the Arduino Uno

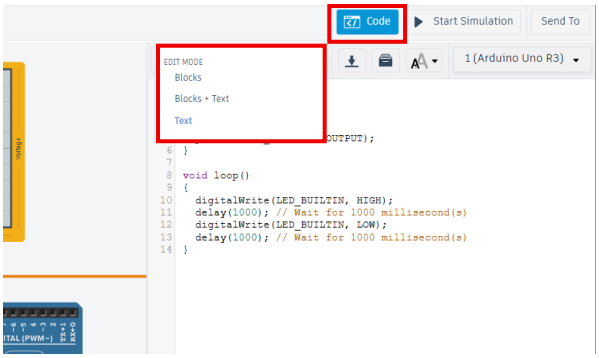


Open the Code Editor

Click on the "Code" in the top right corner next to "Start Simulation." By default, the

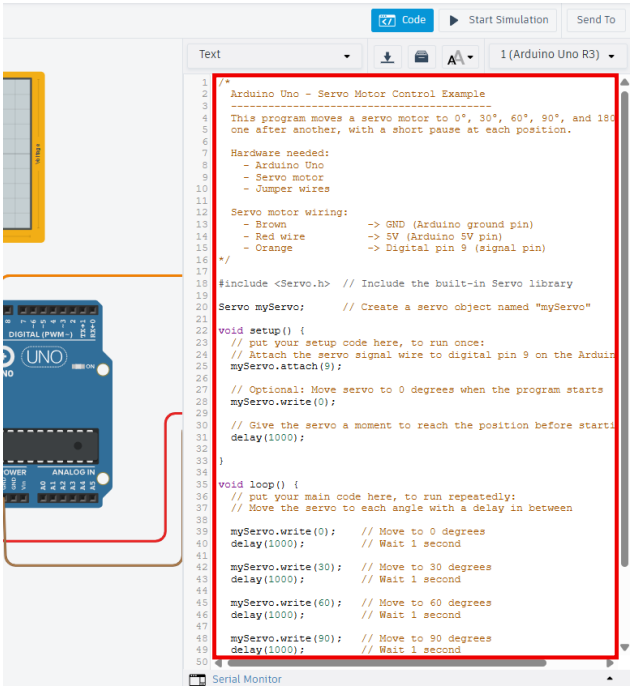


code editor will be in Blocks mode. Select the drop-down menu and click on Text, then continue, to open a text editor.



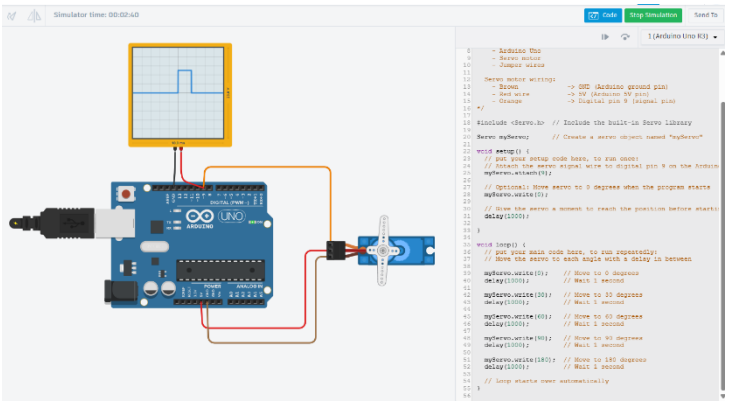
Upload PWM_Servo to the Arduino Uno

Delete the default code in the text editor. Copy the code from the Servo_PWM script created in the previous task and paste it into the text editor in TinkerCAD. TIP: *Adjusting delay(1000) in the loop() function can slow down the delay time between angle changes making it easier to observe. The number entered inside of delay() is how many milliseconds the Arduino Uno will wait before executing the next line of code.*



Run circuit simulation

Press the "Start Simulation" button on the top bar or press spacebar to start the simulation. The Servo will move to angles defined in the code: 0, 30, 60, 90, 180 degrees. Observe the PWM signal measured on the oscilloscope. Notice that the width of the pulse changes to move the motor to a different angle. TIP: *Click and drag in the white space of the simulation workspace to drag the circuit around. Ensure the Oscilloscope, Arduino Uno, and Servo Motor are within view when simulating the circuit.*



Controlling a Motor with an Arduino Uno

In this task, you will investigate how an Arduino Uno can control a servo motor using a Pulse Width Modulation (PWM) signal; one of the most common methods for precise motor control in robotics. Utilizing everything learned from the TinkerCAD simulation tool, you will build a circuit with an Arduino Uno, a servo motor, and an oscilloscope. The oscilloscope will let you visualize the PWM waveform that drives the servo, helping you see the direct link between the signal's characteristics and the motor's movement. This same principle is used in robotic arms, grippers, and steering mechanisms, where accurate position control is essential for reliable robot operation.

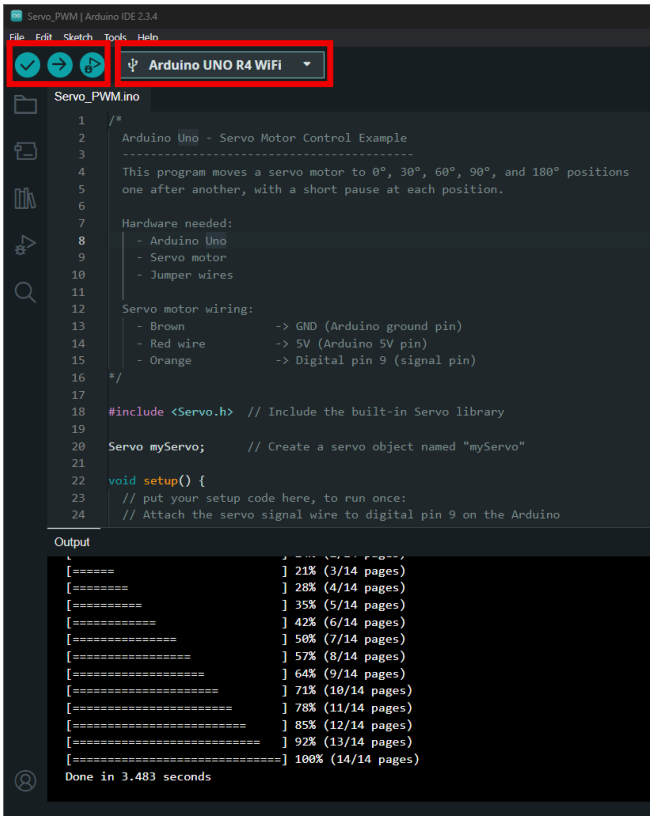
Task: Building Physical Circuit

Materials:

- 1x 2C23T Multimeter + Oscilloscope
- 1x Set of Oscilloscope Alligator Clips
- CrowPi2 w/ Power Cable + Mouse OR personal computer.
- 1x Arduino Uno
- 1x USB C Cable
- 1x Servo Motor
- 6x Jumper Wires

Upload Servo_PWM to the Arduino

Use a USB C cable to connect the Arduino Uno to the CroPi2 or personal computer and open the Arduino Uno IDE. Open the Servo_PWM script made in the previous task make sure "Arduino UNO R4 WiFi" is selected and click the upload button. Once the code is uploaded, unplug USB C cable from the Arduino Uno. TIP: *If the IDE throws a COM error, click on the drop down in the top bar of the IDE and select "Arduino UNO R4 WiFi" again.*

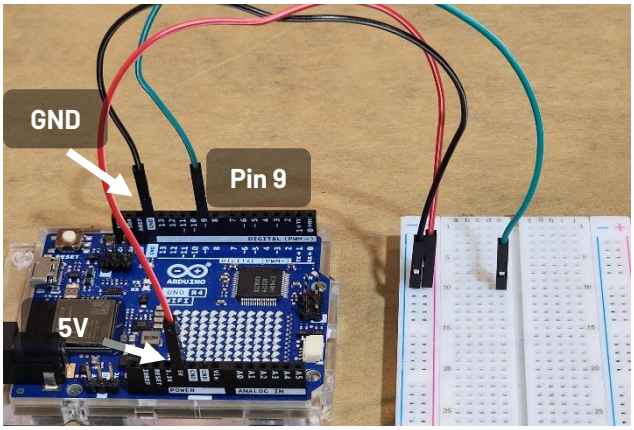


Connect Alligator Clips to Oscilloscope

In the 2C23T Multimeter + Oscilloscope package, there will be a coaxial cable that breaks out into a black (-) and red (+) alligator clip. Connect the coaxial cable to the "CH1" port at the top of the multimeter.

Wire the Arduino

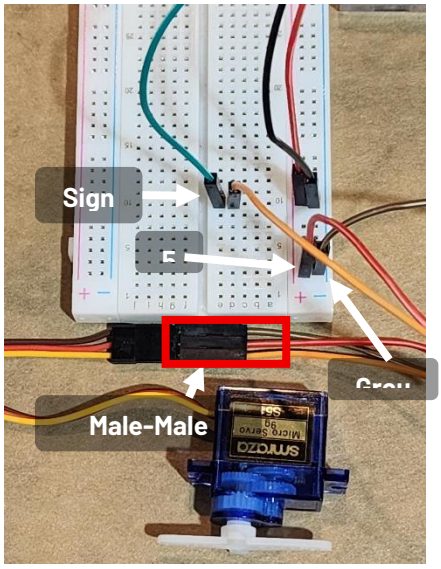
The Arduino will be wired to the breadboard using male-to-male jumper wires:



- Connect Pin 9 on the Arduino to row 10 on the breadboard using a green jumper wire (this sends the PWM signal).
- Connect the pin labeled 5V on the Arduino to the positive power rail on the breadboard using a red jumper wire.
- Connect the pin labeled GND on the Arduino to the negative power rail on the breadboard using a black jumper cable. Review the provide image.

Step 5: Wire the Servo Motor

The servo will be wired to the breadboard using male-to-male jumper wires. The servo motor has three pins. Orange for Signal, Red for Power, and Brown for Ground.



Connect three male-to-male jumper wires to the three available female pins of the Servo Motor so it can be connected to the breadboard. TIP: *Use the same color male-to-male jumper wires when connecting to the female header of the servo motor.*

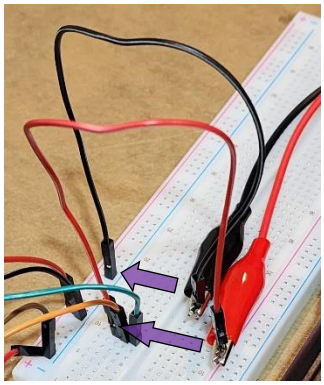
- Connect the orange Signal pin of the servo to row 10 (the same row that Pin 9 of the Arduino is connected to) on to the breadboard

- Connect the red Power pin of the servo to the positive power rail on the breadboard.
- Connect brown Ground pin of the servo motor to the negative power rail on the breadboard.

Connect the 2C23T Multimeter + Oscilloscope

Connect two jumper wires to the alligator clips attached to the multimeter. Use a black jumper wire for the black alligator clip and a red jumper wire for the red alligator clip. The jumper wires will be inserted into the breadboard in the next step to measure the PWM signal output from the Arduino Uno.

Probe the Circuit with the Oscilloscope



Connect the two wires attached to the alligator clips in the previous step to the breadboard to measure between the ground and signal pin.

- Connect the black wire from the Oscilloscope to the ground rail of the breadboard.
- Connect the red wire from the Oscilloscope to row 10 (the same row that Pin 9 of the Arduino is connected to) on to the breadboard.

Turn on the 2C23T Multimeter + Oscilloscope

Power on the 2C23T Multimeter + Oscilloscope with the red power button. To put the 2C23T Multimeter + Oscilloscope in oscilloscope mode, make sure “oscilloscope” is selected and press the enter button. This will bring up a Time vs Voltage plot of the signals being measured. It is currently reading 0V because the Arduino is powered off.

Power on the Arduino & Observe the PWM Signal:

Connect the Arduino to a computer using the USB C cable to power it. The code will begin executing and the servo will begin cycling between the angles specified in the code. Observe the PWM signal on the 2C23T Multimeter + Oscilloscope. TIP: *Make sure the bottom left of the oscilloscope shows 5V. If it doesn't use the up and down arrows to cycle until it is set to 5V. Use the right and left arrows to cycle the total time shown on the plot. This will make the signal easier to see.*

Well Done!

You've completed Module 2: Robotic Electronics. You've gone from exploring the basics of electricity to applying that knowledge in circuits, components, and microcontrollers. More importantly, you've seen how electronic components fit together to transform raw electricity into purposeful action.

By now you should be able to:

- Understand basic electronics within the context of robotics.
- Establish hands-on technical competency to build simple circuits utilized in a robotics context.
- Develop trust in the components comprising a robotics system and their functions within a robot.

Finishing this module means you're no longer just learning about robots from the outside; you now understand what's happening under the hood. This knowledge will help you make sense of more advanced systems, diagnose problems, and trust what your machines are doing in the field.